

Cognitive Robotics: Implementierung einer Befehlswarteschlange und bildbasierter Safety-Funktionalität

Studienarbeit T3200

Studiengang Elektrotechnik
An der Dualen Hochschule Stuttgart
Campus Horb

Eingereicht von:

Name:	Luis Magnus Thiel	Adrian Sommer
Matrikelnr.:	8616207	9519924
Straße, Nr.:	Hugo-Bertsch-Straße 15	Hechinger Straße 40
PLZ, Ort:	72459 Albstadt	72336 Balingen

Partner:	ASSA ABLOY Sicherheitstechnik GmbH	Groz-Beckert KG
Adresse:	Bildstockstraße 20	Parkweg 2
Ort:	72458 Albstadt	72458 Albstadt

Studienjahrgang: TET2023

Bearbeitungszeitraum: 02. März 2026 bis 22. Juni 2026

Betreuer: Norman Bläse (Mercedes-Benz)

Kurzzusammenfassung

Diese Studienarbeit befasst sich mit der Optimierung und Erweiterung eines autonomen Robotersystems, welches aus einem Mitsubishi Movemaster EX RV-M1 und einem Vision-System (2D- & 3D-ToF-Kamera) besteht. Das zentrale Ziel ist dabei die Transformation in ein intelligentes System, um durch die Implementation von bildbasierten Sicherheitsfunktionen einen kollaborativen Betrieb zu evaluieren.

Mittels einer IST-Analyse werden Limitationen der bestehenden Architektur identifiziert und für die nachfolgende Optimierung analysiert. Der Hauptfokus liegt hierbei in der Überführung der synchronen Befehlsstruktur in eine asynchrone Befehlswarteschlange, welche mithilfe der inversen Kinematik die Laufzeit einzelner Befehle errechnet und den Roboter dynamisch ansteuert.

Im Bereich der bildbasierten Safety-Implementation erforscht die Arbeit verschiedene Ansätze zur Handerkennung. Einer dieser Ansätze stellt der Haar Cascade Classifier dar - dieser wurde im Rahmen der Arbeit als geeignetes Verfahren ausgewählt und mittels einer Testreihe geprüft. Aus diesen beiden fundamentalen Änderungen der Steuerungssoftware resultiert eine Basis für die weitere Transformation des Robotersystems hin zum kollaborativen Einsatz.

Abstract

This student research project focuses on the optimisation and extension of an autonomous robotic system comprising a Mitsubishi Movemaster EX RV-M1 and a vision system (2D and 3D ToF cameras). The primary objective is to transform the system into an intelligent one in order to evaluate collaborative operation through the implementation of image-based safety functions.

An ‘as-is’ analysis is used to identify limitations in the existing architecture and analyse them for subsequent optimisation. The main focus here is on converting the synchronous command structure into an asynchronous command queue, which uses inverse kinematics to calculate the execution time of individual commands and dynamically controls the robot.

In the field of image-based safety implementation, this thesis investigates various approaches to hand detection. One such approach is the Haar Cascade Classifier – this was selected as a suitable method within the scope of the thesis and tested through a series of trials. These two fundamental changes to the control software provide a basis for the further transformation of the robot system towards collaborative use.

Selbständigkeitserklärung

Hiermit erklären wir, Luis Magnus Thiel und Adrian Sommer, dass wir diese schriftliche Studienarbeit selbständig verfasst haben. Es wurden keine anderen als die angegebenen Hilfsmittel und Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen sind als solche gekennzeichnet.

Horb, den 22. Juni 2026

Ort, Datum



Unterschrift (Luis Magnus Thiel)

Horb, den 22. Juni 2026

Ort, Datum



Unterschrift (Adrian Sommer)

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	2
2.1	Stand der Technik: Safety-Implementation & Retrofitting .	2
2.1.1	kollaborative Robotik	2
2.1.2	Visuelle Handerkennung	4
2.1.2.1	Infrarot- und Wärmebildsensorik	4
2.1.2.2	2D- und 3D-Time-of-Flight-Scanner	5
2.1.2.3	Hautfarbenextraktion	6
2.1.3	Normen und Richtlinien	7
2.1.3.1	Rechtliche Grundlagen und funktionale Sicherheit	7
2.1.3.2	Sicherheitsanforderungen an kollaborative Robotersysteme	8
2.1.3.3	Sicherheitsgerichtete Anforderungen an optische Schutzsysteme	8
2.1.4	Cascade classifier	9
2.1.4.1	Haar-like Features	9
2.1.4.2	Mathematische Grundlagen und algorithmische Optimierung der Haar-Merkmale .	11
2.2	Bildverarbeitung	13
2.2.1	Gaußfilter	13
2.2.2	Grayfilter	14
2.2.3	Threshold von Graustufenbildern	14
2.3	Grundlagen der Informatik	14
2.3.1	Producer-Consumer-Pattern	14
2.3.2	Task Parallel Library (TPL)	15
2.3.2.1	Task-based Asynchronous Pattern (TAP) .	16
2.3.2.2	Task Cancellation/kooperativer Abbruch .	17
2.4	Inverse Kinematik des Mitsubishi Movemaster EX RV-M1	17
2.4.1	Kinematische Parameter des RV-M1	17
2.4.2	Trigonometrische Kompaktnotation nach [1]	18
2.4.3	Analytische Gelenkwinkelberechnung	19
3	Ausgangssituation/IST-Analyse	21
3.1	Architekturlimitationen des bisherigen Systems	21
3.1.1	Verwaltung der Punktwolken-Daten	21

3.1.2	Sequenzielle Befehlsstruktur	22
3.1.2.1	Theoretische Limitation	23
3.1.2.2	Messexperiment	24
3.1.2.3	Praktische Limitation - Laufzeitmessung mit Simulationsexperiment	25
4	Architekturoptimierung und -erweiterung	28
4.1	Konzeptionierung der dynamischen Robotersteuerung . .	28
4.1.1	Optionen zur Positionserkennung/Befehl- Bewegungssynchronisation	28
4.2	Befehlswarteschlange für dynamische Robotersteuerung	31
5	Implementation der Erweiterungen	33
5.1	Safetyimplementation	33
5.1.1	Kriterienbasierte Evaluierung der Sicherheitsüber- wachungssysteme	33
5.1.2	Machine-learning Modell zur visuellen Handerken- nung	38
5.1.2.1	Machine learning-Modell	38
5.1.2.2	Haar Cascade Classifier	39
5.1.2.3	Testreihe	40
5.2	Befehlspipeline	45
5.2.1	Vergleich von theoretischer und realer Performance	45
5.3	Integration der Safety-Implementation in die Befehlspipe- line	46
5.3.1	Kooperativer Abbruchmechanismus mit Cancellat- ionTokenSource	46
6	Fazit & Ausblick	48
	Literaturverzeichnis	vii
	Abkürzungsverzeichnis	x
	Abbildungsverzeichnis	xiii
	Erklärungen zur Arbeit und Hilfsmitteln	xiv

1 Einleitung

Die vorliegende Studienarbeit T3200 mit dem Titel *Cognitive Robotics: Implementierung einer Befehlswarteschlange und bildbasierter Safety-Funktionalität* befasst sich im ersten Schritt mit der fundierten Erarbeitung der benötigten methodischen und technologischen Grundlagen. Darauf aufbauend erfolgt eine detaillierte Analyse der Ausgangslage, welche explizit an die Ergebnisse und den Stand der vorherigen Studienarbeit anknüpft. Bestandteil dieser Evaluation sind eine umfassende Ist- und Laufzeitanalyse sowie die Ausarbeitung und Umsetzung daraus abgeleiteter Optimierungsvorschläge.

Die Umsetzung der Optimierungsvorschläge lässt sich dabei zentral in zwei Hauptziele einteilen. Bei ersterem handelt es sich um die Entwicklung und Implementation einer asynchronen Befehlspipeline zur Optimierung und Stabilisierung der Systemlaufzeit. Ein weiterer Aspekt dieser Pipeline wird durch die Möglichkeit gegeben, die Befehlsabfolge gezielt zu unterbrechen, um ein Stoppen beim Eingriff eines Fremdobjekts in den Operationsbereich des Roboters zu realisieren.

Die Erkennung der Fremdobjekte wird dabei durch das zweite Hauptziel der Arbeit näher thematisiert. Der Fokus liegt dabei auf der bildbasierten Erfassung einer menschlichen Hand in einem fest definierten Sicherheitsbereich. Hierfür werden zunächst gängige Methoden evaluiert und bewertet. Darauf aufbauend wird der Ansatz des *Haar Cascade Classifier* am Versuchsaufbau getestet sowie die Integration in die finale Steuerungsarchitektur realisiert.

2 Grundlagen

2.1 Stand der Technik: Safety-Implementation & Retrofitting

In diesem Kapitel werden alle relevanten Fakten des Retrofittings im Kontext der Sicherheitsimplementation beleuchtet.

2.1.1 kollaborative Robotik

Für die Modernisierung älterer Industrie- und Robotersysteme (Retrofitting) im Hinblick auf aktuelle Sicherheitsstandards und moderne Sensortechnik existieren vielfältige technologische Ansätze. Diese lassen sich anhand verschiedener Systemparameter kategorisieren. Ein grundlegendes Unterscheidungsmerkmal ist die Dimensionierung der Bilderfassung, bei der entweder kostengünstige 2D-Kameras oder tiefenauflösende 3D-Kamerasysteme eingesetzt werden. Zudem variiert die Systemarchitektur hinsichtlich der Kameraanzahl: Während Monokamerasysteme eine einfache Raumüberwachung ermöglichen, bieten Multikamerasysteme durch hardwareseitige Redundanz eine deutlich verlässlichere Objekterkennung sowie Schutz vor Verdeckungen im sensornahen Bereich.

Die algorithmische Auswertung lässt sich wiederum in zwei Philosophieansätze unterteilen: die gezielte Klassifizierung spezifischer Objekte wie menschlicher Gliedmaßen oder die universelle Erfassung jeglicher Fremdobjekte innerhalb eines definierten Raumbereichs. Ergänzend oder alternativ zu visuellen Kamerasystemen kommen in der Praxis berührungslose Sensoren auf kapazitiver, induktiver oder laserbasierter Basis (z.B. Laserscanner) zum Einsatz.

Die informationstechnische Absicherung des Arbeitsraumes erfolgt je nach Systemkomplexität über die Festlegung einer einzigen, starren Verbotszone oder über die Aufteilung in mehrere, flexible Zonen. Letzteres ermöglicht die Implementierung einer dynamischen Geschwindigkeitsregelung (Dynamic Speed Control), bei der die Systemgeschwindigkeit adaptiv an die Position des Menschen angepasst wird. Diese Verbots- und Schutzräume können entweder statisch im Raum verankert oder dynamisch an die aktuelle Kinematik

sowie den Bremsweg der Anlage gekoppelt werden, wie es unter anderem in Konzepten des Fraunhofer-Instituts [2] realisiert wird.

Ein zentrales wissenschaftliches Fundament für solche flexiblen Sicherheitsarchitekturen beschreiben Bdiwi, M., Pfeifer, M. und Sterzing, A. [3], die sich intensiv mit unterschiedlichen Aufgabenszenarien der Mensch-Roboter-Kollaboration (MRK) befassen. Die Autoren schlagen eine systematische Taxonomie vor, welche die Interaktion zwischen Mensch und Maschine in vier aufeinander aufbauende Stufen gliedert:

**Level 1: Geteilter Raum
(Sicherheitsgerichteter Stopp)**

**Level 2: Gemeinsame Aufgabe
(Speed & Separation Monitoring)**

**Level 3: Handing-Over
(Hybride Steuerung / Dynamik)**

**Level 4: Physische Interaktion
(Kraftregelung / Guiding)**

- **1. Geteilter Arbeitsraum:** In diesem Szenario arbeiten Mensch und Roboter räumlich nah beieinander, führen jedoch strikt getrennte Aufgaben in exakt zugewiesenen Zonen aus. Sobald der Mensch die virtuelle Grenze zur Roboterzone überschreitet, detektiert die Sensorik eine Schutzraumverletzung und löst unverzüglich einen Sicherheitsstopp der Anlage aus.
- **2. Gemeinsame Aufgabe ohne physischen Kontakt:** Hier agieren beide Akteure innerhalb einer vordefinierten Kooperationszone an einem gemeinsamen Prozess. Anstelle eines abrupten Halts wird die Geschwindigkeit des Roboters in Abhängigkeit vom realen Abstand zum Menschen kontinuierlich gedrosselt.
- **3. Bauteilübergabe (Handing-over):** Dieses Level beschreibt den direkten Transfer von Werkzeugen oder Werkstücken zwischen Mensch und Roboter. Eine hybride Steuerung sorgt in dieser Phase dafür, dass der Manipulator dynamisch und feinfühlig auf die Handbewegungen des Menschen im freien Raum reagiert.

- **4. Gemeinsame Aufgabe mit physischer Interaktion:** Auf der höchsten Kooperationsstufe ist ein direkter Körperkontakt zwingend erforderlich. Ein Praxisbeispiel hierfür ist die manuelle Führung eines Schwerlastroboters durch menschliche Handkräfte, um schwere Bauteile präzise zu positionieren.

Die technische Umsetzung der Überwachung innerhalb dieser Interaktionsstufen stützt sich im Wesentlichen auf drei Säulen: die permanente Überwachung der internen Roboterparameter (wie Position, Achsgeschwindigkeiten und Kräfte), eine automatisierte Ereigniserkennung zur Identifikation kritischer Zustände (z.B. die visuelle Verdeckung eines Menschen im Sensorschatten) sowie eine kamerabasierte Nahfelderkennung. Letztere dient dazu, im direkten Umfeld des Endeffektors sowohl die sensorische Bereitschaft des Bedieners (z.B. via Gesichtserkennung) als auch die exakte Lage von Körperteilen zu erfassen, um Verletzungen oder Quetschungen effektiv zu verhindern.

2.1.2 Visuelle Handerkennung

In diesem Kapitel werden verschiedene informationstechnische Ansätze, Methoden und Praktiken zur visuellen beziehungsweise optischen Handerkennung detailliert beleuchtet. Die theoretische Auseinandersetzung mit diesen technologischen Konzepten ist im Rahmen dieser Studienarbeit von zentraler Bedeutung, um die dedizierte Auswahl des in dieser Arbeit implementierten Verfahrens wissenschaftlich und methodisch zu begründen. Zu diesem Zweck werden im Folgenden die Vor- und Nachteile der jeweiligen Herangehensweisen systematisch evaluiert.

2.1.2.1 Infrarot- und Wärmebildsensorik

Ein etablierter Ansatz zur Detektion des menschlichen Körpers basiert auf der Nutzung von Passiv-Infrarot-Sensoren (PIR), welche im infraroten Spektralbereich operieren. Diese Systeme messen die emittierte Wärmestrahlung von Objekten, die sich innerhalb des definierten Erfassungsbereichs befinden. Mittels spezieller Lernalgorithmen, die auf verschiedenen empirischen Fallstudien basieren, gelingt auf dieser Grundlage die selektive Erkennung menschlicher, warmer Haut [4].

Als wesentlicher Vorteil dieses sensorischen Prinzips erweist sich die robuste Einsetzbarkeit in rauen, industriell geprägten Produktionsumgebungen, da das Messprinzip weitgehend unabhängig von sichtbaren transienten Lichtverhältnissen arbeitet.

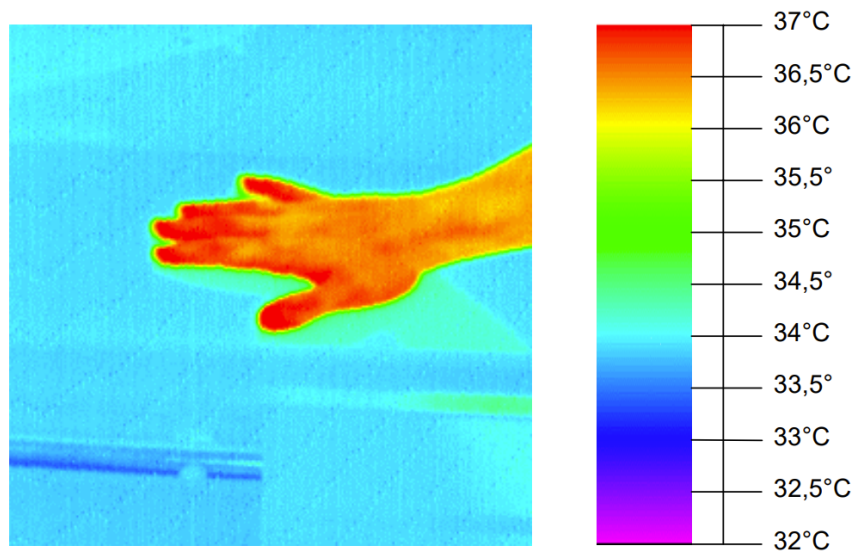


Abbildung 2.1: Testbild Passiv-Infrarot Schutzeinrichtung an einer Kreissäge aus [4]

Demgegenüber stehen jedoch signifikante Nachteile: Neben der Notwendigkeit kostenintensiver Infrarotsensoren neigt das System zu Fehl- und Pseudoerkennungen, sobald die Temperatur umliegender Maschinenteile, Werkstücke oder Betriebsmedien der menschlichen Hautoberflächentemperatur ähnelt. Darüber hinaus erfordert die thermografische Signalverarbeitung und die nachgelagerte algorithmische Merkmalsberechnung eine vergleichsweise hohe Rechenleistung.

2.1.2.2 2D- und 3D-Time-of-Flight-Scanner

Andere wissenschaftliche Arbeiten, wie die Forschungen von Wolfgang Schulz [5], behandeln die Detektion von Handkonturen primär auf Basis von Abstandsaufnahmen. Technologisch lässt sich dieses Prinzip entweder als „virtueller Vorhang“ mittels einer planar messenden 2D-Time-of-Flight-Kamera (ToF) [5] oder als volumetrischer Scan des kompletten Arbeitsbereichs mithilfe einer 3D-ToF-Kamera realisieren, wie es unter anderem von Mandischer et al. [6] demonstriert wurde. Ein solches volumetrisches System ist in der Lage, einen großflächigen Arbeitsbereich von etwa 2×2 m lückenlos abzudecken.

Mithilfe der ToF-Struktur wird kontinuierlich die euklidische Distanz zwischen der Roboterkinematik und den im Raum befindlichen Objekten ermittelt. Unterschreitet der gemessene Abstand zwischen dem Manipulator und einem externen Objekt normative

Schwellenwerte, greift ein algorithmisches Geschwindigkeits- und Abstandsmodell (*Speed and Separation Monitoring*, SSM), um den Roboter kontrolliert zu verlangsamen oder vollständig stillzusetzen.

Die Vorteile dieser Technologie liegen in der Option, eine mitbewegte Sensorik zu implementieren; das Kamerasystem kann hierbei direkt an der mechanischen Struktur des Roboters fixiert werden und wandert mit dem Gefahrenbereich mit. Zudem ermöglicht die inhärente Kombination aus Intensitätsbildern und dreidimensionalen Tiefendaten eine zuverlässige Positionsermittlung sowie die Differenzierung zwischen einem maschinellen Werkstück und einem menschlichen Körperteil.

Als kritischer Nachteil erweist sich jedoch die normative Zertifizierung im Kontext der funktionalen Sicherheit. Damit eine ToF-Kamera als berührungslos wirkende Schutzeinrichtung (ESPE) zugelassen werden kann, erfordert die Systemarchitektur eine hardwarenahe Zweikanaligkeit (Redundanz) sowie eine permanente, fehlersichere Selbstüberwachung. Zudem bedingt die hochfrequente Verarbeitung großer 3D-Punktwolken einen erheblichen Rechenaufwand, was zu systemischen Latenzzeiten in der Sicherheitskette führen kann.

2.1.2.3 Hautfarbenextraktion

Eine alternative Methode stellt die Extraktion und Erkennung menschlicher Körperteile auf Basis von rein farbmétrischen Informationen dar. Entsprechende Algorithmen nutzen Schwellwertverfahren, Histogrammanalysen oder vordefinierte Look-up-Tables (LUT), um bestimmte Farbwerte im Bild zu isolieren. Diese Analysen werden in unterschiedlichen Farbräumen wie RGB, HSV oder YCbCr durchgeführt. Ein kombinierter Ansatz, welcher die mathematischen Vorteile der Farbräume RGB, HSV und YCbCr kombiniert, wird in [7] beschrieben. Durch den sequentiellen Einsatz verschiedener Spektralfilter und komplexer Segmentierungsalgorithmen kann auf diese Weise eine robuste Extraktion der Handgeometrie oder anderer freiliegender Extremitäten erreicht werden.

Der fundamentale Vorteil dieser Methode liegt in der Hardware-Ökonomie, da für die Umsetzung eine klassische, kostengünstige 2D-Industriekamera ausreichend ist. Zudem arbeitet die farbbasierte Segmentierung weitgehend unabhängig von der konkreten Bewegung, Rotation oder temporären geometrischen Verformung der menschl-

chen Hand.

Dem stehen im industriellen Einsatz gravierende Nachteile gegenüber. Das System versagt inhärent, sobald der Werker vorgeschriebene Arbeitshandschuhe trägt, da die Farbinformation der Haut maskiert wird. Zudem verursachen die mathematischen Transformationen zwischen den Farbräumen und die nachgelagerte Segmentierung einen hohen theoretischen sowie rechentechnischen Aufwand. Das Verfahren setzt qualitativ hochwertige Kamerasysteme voraus und erfordert eine präzise Kalibrierung auf das jeweilige optische System. Zuletzt begründet die strikte Abhängigkeit von den visuellen Hintergrundverhältnissen sowie wechselnden Beleuchtungsszenarien ein hohes Risiko für Falschdetektionen im praktischen Betrieb.

2.1.3 Normen und Richtlinien

Die Auslegung von Systemen der Mensch-Roboter-Kollaboration (MRK) sowie der Einsatz optischer Schutzsysteme unterliegen einem strikten regulatorischen Rahmen. Da bei kollaborativen Arbeitsplätzen die klassischen physisch-trennenden Schutzeinrichtungen (wie Schutzzäune) entfallen, muss die Sicherheit über sensorisch aufgenommene und durch spezielle Logik- verarbeitete Daten gewährleistet werden.

2.1.3.1 Rechtliche Grundlagen und funktionale Sicherheit

Die primäre rechtliche Basis für das Inverkehrbringen sowie wesentliche Veränderungen (Retrofitting) von Maschinenanlagen im europäischen Raum bildet die **Maschinenrichtlinie 2006/42/EG** [8] (bzw. die EU-Maschinenverordnung 2023/1230) [9]. Sie verpflichtet den Systemintegrator zu einer umfassenden Risikobeurteilung. Die steuerungstechnische Umsetzung des Sicherheitsnachweises wird durch die **DIN EN ISO 13849-1** [10] definiert. Diese Norm klassifiziert die Zuverlässigkeit sicherheits-bezogener Teile von Steuerungen anhand von fünf Performance Levels (PL_a bis PL_e). Für industrielle Roboterzellen im kooperativen Betrieb wird im Regelfall ein Mindest-Sicherheitsintegritätslevel von **PL d (Kategorie 3)** gefordert. Dies bedingt eine zweikanalige Struktur (Hardware-Redundanz) um, wie bereits in der vorherigen Studienarbeit erwähnt, die geforderte Ein-Fehler-Sicherheit zu gewährleisten.

2.1.3.2 Sicherheitsanforderungen an kollaborative Robotersysteme

Die allgemeinen Sicherheitsanforderungen für Industrieroboter sind in der Normenreihe **DIN EN ISO 10218-1/-2** [11] [12] verankert. Normativ werden vier Schutzprinzipien für die Mensch-Roboter-Interaktion definiert:

1. Sicherheitsgerichteter überwachter Halt (*Safety-rated monitored stop*)
2. Handführung (*Hand guiding*)
3. Geschwindigkeits- und Abstandsüberwachung (*Speed and Separation Monitoring, SSM*)
4. Kraft- und Leistungsbegrenzung (*Power and Force Limiting, PFL*)

Beim Einsatz berührungsloser Raumüberwachungssysteme ist das Prinzip des Speed and Separation Monitoring (SSM) von zentraler Bedeutung. Die **ISO/TS 15066** [13] liefert hierzu die mathematische Grundlage zur Berechnung des Mindestsicherheitsabstands S_p zwischen Mensch und Roboter:

$$S_p = v_H \cdot (T_R + T_B) + v_R \cdot T_R + s_R + C \quad (2.1)$$

In diese Berechnung fließen die menschliche Annäherungsgeschwindigkeit v_H , die Systemlatenzzeiten (Reaktionszeit der Sensorik T_R und Bremszeit des Roboters T_B) sowie gerätespezifische Nachlaufwege s_R und sensorische Zuschläge C ein.

2.1.3.3 Sicherheitsgerichtete Anforderungen an optische Schutzsysteme

Wird der Schutzraum volumetrisch überwacht, greift die Normenreihe **DIN EN IEC 61496-1** [14], welche die Anforderungen an berührungslos wirkende Schutzeinrichtungen (ESPE) regelt. Für den Einsatz in sicherheitsgerichteten Applikationen müssen optische Systeme als ESPE des Typs 3 oder Typs 4 klassifiziert sein. Dies setzt voraus, dass das Sensorsystem über eine permanente, hardwarenahe Selbstüberwachung (z.B. zur Erkennung von Linsenverschmutzungen, Signalverlusten oder internen Rechenfehlern) verfügt. Standardmäßige, einkanale Industriekameras ohne inhärente Redundanz sind für die Einleitung sicherheitsgerichteter Reaktionen normativ unzulässig.

Ergänzend definiert die **DIN EN ISO 13855** [15] die Anordnung von

Schutzeinrichtungen im Hinblick auf die Annäherungsgeschwindigkeit des menschlichen Körpers. Sobald ein System auf die Detektion von Teilbereichen des Körpers, wie Hände oder Finger, ausgelegt wird, fordert die Norm den Ansatz standardisierter, maximaler Handgeschwindigkeiten (1600 mm/s bis 2000 mm/s). Für optische Systeme, die auf Mustererkennung basieren, ergibt sich daraus theoretisch die Anforderung, dass die Verarbeitungsfrequenz (*Frames per Second*) und die Pipeline-Latenz so dimensioniert sein müssen, dass eine Handbewegung innerhalb dieser Dynamikgrenzen deterministisch und ohne unzulässige Zeitverzögerung erfasst werden kann.

Zusammenfassend ergibt sich in der mathematisch-deterministischen Validierung nach ISO 13849-1 [10] und IEC 61496 [14] eine theoretische Diskrepanz beim Einsatz stochastischer Auswertealgorithmen (wie maschinellen Lernverfahren). Da diese systembedingt keine klassischen deterministischen Zustandstabellen aufweisen, fordert die normative Praxis bei sicherheitskritischen Anwendungen stets eine Absicherung der mathematischen Ergebnisse durch diversitäre Sensorik oder fest programmierte Hardware-Gateways.

2.1.4 Cascade classifier

Die fundamentale Aufgabe der Haar-Cascade-Methode liegt in der effizienten Erkennung und Extraktion spezifischer Merkmale (*Features*) innerhalb einer digitalen Bildmatrix. [16] Diese bildbasierten Merkmale lassen sich dabei mathematisch und visuell anhand von grundlegenden Eigenschaften wie der Farbe, der geometrischen Form oder der absoluten Größe des zu detektierenden Objekts identifizieren und abgrenzen. [17] Um diese Erkennung prozesstechnisch umzusetzen, stützt sich die Architektur des Kaskadenklassifikators (*Cascade Classifier*) auf eine sequentielle, hierarchisch strukturierte Liste von aufeinanderfolgenden Stufen (*Stages*). Jede dieser einzelnen Stufen fungiert als separater Filter innerhalb der Kontrollkette und setzt sich wiederum aus einer definierten Liste von sogenannten schwachen Lernern (*Weak Learners*) zusammen, welche in ihrer Gesamtheit eine hocheffiziente und rechenschonende Entscheidungsmatrix bilden. [16]

2.1.4.1 Haar-like Features

Die Objektdetektion mittels eines Kaskadenklassifikators basiert auf einem sequenziellen Suchfenster-Prinzip, bei dem ein vordefiniertes Fenster (wie in Abbildung 2.2) schrittweise über die gesamte Bildmatrix

verschoben wird. Innerhalb dieses Prozesses führt jede einzelne Stufe der Kaskade eine binäre Klassifikation der aktuell fokussierten Bildregion durch, um diese entweder als potenzielles Objekt (positiv) oder als Hintergrund (negativ) einzustufen. Der logische Ablauf dieser Klassifikationskette folgt dabei einem strikten Ausschlussverfahren: Sobald eine Stufe ein negatives Ergebnis liefert, wird die Analyse der aktuellen Region unverzüglich abgebrochen und das Suchfenster rückt zur nächsten Position weiter. Ein positives Resultat hingegen bewirkt, dass der Bildausschnitt zur tiefergehenden Überprüfung an die nächsthöhere Stufe ($n + 1$) übergeben wird. Ein Objekt gilt folglich erst dann als final und verlässlich detektiert, wenn es sämtliche Stufen der Kaskadenstruktur erfolgreich und durchgehend positiv durchlaufen hat. [16] Zur Bewertung der Klassifikationsgüte dieses Systems lässt

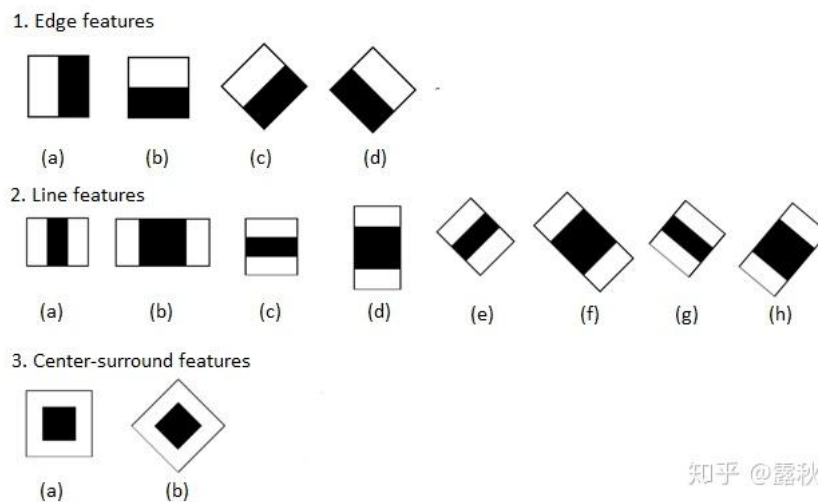


Abbildung 2.2: Prinzip-Skizze Haar-like-features aus [18]

sich das Prinzip der Konfusionsmatrix heranziehen, welche die Ergebnisse in vier fundamentale Kategorien unterteilt. Ein *True Positive* (Richtig-Positiv) beschreibt dabei die korrekte Erkennung eines real existierenden Objekts, während ein *True Negative* (Richtig-Negativ) die zutreffende Identifikation von Hintergrundstrukturen als Nicht-Objekt darstellt. Kritischer sind die Fehlklassifikationen: Ein *False Positive* (Falsch-Positiv) liegt vor, wenn ein Hintergrundbereich fälschlicherweise als Zielobjekt eingestuft wird, wohingegen ein *False Negative* (Falsch-Negativ) bedeutet, dass ein tatsächlich vorhandenes Objekt vom Algorithmus übersehen wird.

Aus dieser Dynamik leiten sich hochgradig asymmetrische und sicherheitskritische Anforderungen an die einzelnen Kaska-

denstufen ab. Da ein einmalig als negativ deklarierter Bildbereich sofort verworfen wird, ist eine extrem niedrige Falsch-Negativ-Rate pro Stufe zwingend erforderlich; ein hier entstandener Fehler lässt sich im späteren Verlauf der Detektion nicht mehr korrigieren. Im Gegensatz dazu ist eine höhere Falsch-Positiv-Rate in den einzelnen Stufen durchaus tolerierbar. Falls eine frühe Instanz der Kaskade ein Nicht-Objekt fälschlicherweise als positiv bewertet und passieren lässt, verbleibt für das Gesamtsystem die Möglichkeit, diesen Irrtum in den nachfolgenden, spezialisierteren Filterstufen ($n + 1$) nachträglich zu korrigieren und die Fehlleitung zu eliminieren.

2.1.4.2 Mathematische Grundlagen und algorithmische Optimierung der Haar-Merkmale

Die mathematische Zusammensetzung der klassischen Objektdetektion nach Viola und Jones basiert auf der Definition standardisierter, rechteckiger Merkmalsmasken. Der Algorithmus spezifiziert standardmäßig fünf grundlegende Typen von Helligkeitsmasken, deren feste Breiten-Höhen-Seitenverhältnisse als

- (a) 2 : 1
- (b) 3 : 1
- (c) 1 : 2
- (d) 1 : 3
- (e) 2 : 2

definiert sind. Diese geometrischen Vorgaben bedingen strikte Dimensionseinschränkungen bei der Skalierung, so muss beispielsweise bei einem Verhältnis von 3 : 1 die absolute Breite der Maske zwingend ein Vielfaches von drei betragen.

Durch das systematische Verschieben und kontinuierliche Skalieren dieser fünf Basis-Rechtecktypen über die Bildmatrix entsteht eine kombinatorische Explosion der zu berechnenden Werte. Bereits innerhalb eines sehr kleinen Standard-Suchfensters von lediglich 24×24 Pixeln generieren die Masken insgesamt 160.381 unterschiedliche Merkmalswerte. Diese enorme Vielfalt teilt sich analytisch in x Merkmale für die jeweiligen Typen auf:

- (a) 43.200 Merkmale
- (b) 27.600 Merkmale
- (c) 43.200 Merkmale
- (d) 27.600 Merkmale
- (e) 20.736 Merkmale

Für ein nicht-rotiertes Bild der Dimension $W \times H$ und ein minimales Basismerkmal der Größe $w \times h$ lässt sich die exakte Anzahl der generierten Merkmale über die Skalierungsfaktoren $X = W/w$ und $Y = H/h$ durch folgende analytische Formel bestimmen:

$$\text{Anzahl} = XY \left[W + 1 - \frac{w}{2}(X + 1) \right] \left[H + 1 - \frac{h}{2}(Y + 1) \right] \quad (2.2)$$

Um auch Objekte mit veränderter Orientierung im Raum zu erfassen, existiert eine Erweiterung für rotierte Merkmalsrechtecke. Bei dieser geometrischen Anpassung müssen die Breiten- und Höhenparameter mathematisch transformiert und das funktionale Verhältnis anhand der veränderten Dimensionen neu ausgerichtet werden. Unter Einbeziehung der Hilfsvariable $z = w + h$ sowie den angepassten Skalierungsfaktoren $X = W/z$ und $Y = H/z$ berechnet sich die modifizierte Anzahl der rotierten Features nach der Formel:

$$\text{Anzahl (rotiert)} = XY \left[W + 1 - \frac{z}{2}(X + 1) \right] \left[H + 1 - \frac{z}{2}(Y + 1) \right] \quad (2.3)$$

Da die direkte Berechnung dieser immensen Merkmalsanzahl die Rechenzeit kritisch maximieren würde, wird zur algorithmischen Optimierung das Konzept des Integralbildes – auch als Summenflächentabelle (*Summed-Area Table*, SAT) bezeichnet – implementiert. Mathematisch ist der Wert an einem beliebigen Koordinatenpunkt $ii(x, y)$ des Integralbildes als die akkumulierte Summe aller Graustufenwerte derjenigen Pixel definiert, die sich in dem Rechteck von der linken oberen Bildecke $(0, 0)$ bis zu eben diesem Punkt erstrecken. Die Generierung dieses Integralgraphen erfolgt deterministisch und hocheffizient durch ein einmaliges Scannen der Bildmatrix mittels einer zweistufigen, zeilen- und spaltenweisen Rekursionsformel:

$$s(i, j) = s(i, j - 1) + f(i, j) \quad \text{mit} \quad s(i, -1) = 0 \quad (2.4)$$

$$ii(i, j) = ii(i - 1, j) + s(i, j) \quad \text{mit} \quad ii(-1, j) = 0 \quad (2.5)$$

Der entscheidende Vorteil dieses Verfahrens zeigt sich bei der anschließenden mathematischen Flächenberechnung innerhalb der Detektionspipeline. Mithilfe des vorbereiteten Integralbildes kann die Summe der Pixelintensitäten einer beliebigen, isolierten rechteckigen Region D über eine einfache 4-Punkt-Formel ermittelt werden. Unter Verwendung der vier Eckpunkte 1 (oben links), 2 (oben rechts), 3 (unten links) und 4 (unten rechts) reduziert sich die Berechnung auf lediglich vier gezielte Tabellenzugriffe:

$$\text{rectsum}(D) = ii(4) + ii(1) - ii(2) - ii(3) \quad (2.6)$$

Da diese Operation unabhängig von der tatsächlichen Größe des Bildbereichs oder der Skalierungsstufe des Merkmals stets eine konstante Anzahl an Zugriffen erfordert, sinkt die rechentechnische Komplexität drastisch. Da die gesamte Pipeline somit fast ausschließlich auf einfachen Grundrechenarten wie der Addition und Subtraktion basiert, resultiert daraus eine bemerkenswert niedrige Rechenleistung. Dies begünstigt eine hardwarenahe Ausführung in Echtzeit, ohne das System auf kostenintensive grafische Beschleuniger anzuweisen. [18]

2.2 Bildverarbeitung

Dieses Kapitel erläutert kurz gebündelt, die Funktionsweise der verwendeten Filter für die Bildverarbeitung.

2.2.1 Gaußfilter

Der Gauß-Filter (oft auch als Gauß-Weichzeichner oder Gaussian Blur bezeichnet) ist ein lineares Filterverfahren in der Bildverarbeitung, das primär zur Reduzierung von hochfrequentem Bildrauschen und zur Glättung von Pixelstrukturen eingesetzt wird. Der Algorithmus berechnet den neuen Farb- oder Graustufenwert eines Pixels über den gewichteten Durchschnitt seiner Nachbarpixel, wobei die Gewichtung auf einer zweidimensionalen Gauß-Verteilung basiert. Dadurch besitzen Bildpunkte im direkten Zentrum der Maske die höchste mathematische Relevanz, während der Einfluss umliegender Pixel mit zunehmendem Abstand exponentiell abnimmt. Technisch wird diese Operation durch die Faltung des Bildes mit einer quadratischen Matrix, dem sogenannten Gauß-Kernel, realisiert, dessen Glättungsintensität über die Standardabweichung σ (Sigma) präzise gesteuert werden kann. Da das Verfahren feine Sensorstörungen effektiv eliminiert, die Hauptkanten des Objekts jedoch im Wesentlichen erhält, bildet es eine fundamentale Grundlage für die nachgeschaltete Merkmalerkennung [19].

2.2.2 Grayfilter

Der Graustufenfilter ist ein grundlegendes Bildverarbeitungsverfahren, das die dreidimensionale Farbmatrix eines Bildes (meist im RGB-Format) in eine eindimensionale Luminanzmatrix überführt. Der Algorithmus reduziert den Rechenaufwand nachfolgender Merkmalerkennungen drastisch, da statt dreier Farbkanäle für Rot, Grün und Blau nur noch ein einziger Helligkeitswert pro Pixel verarbeitet werden muss. Durch diesen algorithmischen Schritt verliert das Bild zwar jegliche Farbinformation, behält jedoch alle für die Kantendetektion und Kontrastanalyse relevanten Geometrie- und Texturmerkmale bei [20].

2.2.3 Threshold von Graustufenbildern

Die Threshold-Methode von OpenCV ist ein fundamentales Verfahren zur Bildbinnarisierung, das ein Graustufenbild in ein reines Schwarz-Weiß-Bild überführt. Der Algorithmus analysiert jeden einzelnen Pixelwert der Graustufenmatrix und vergleicht diesen mit einem vordefinierten Schwellenwert (Threshold). Liegt der Helligkeitswert des aktuellen Pixels über diesem Schwellenwert, wird ihm mathematisch der maximale Intensitätswert (üblicherweise 255 für Weiß) zugewiesen, andernfalls wird er auf den Minimalwert (0 für Schwarz) gesetzt. Durch diese binäre Segmentierung wird das Zielobjekt trennscharf von seinem Hintergrund isoliert, wodurch nachfolgende Konturen- und Formanalysen erheblich erleichtert werden. Bibliotheken wie OpenCV stellen „ready to use“-Lösungen zur Verfügung [21].

2.3 Grundlagen der Informatik

2.3.1 Producer-Consumer-Pattern

Das *Producer-Consumer-Pattern* ist ein Entwurfsmuster der Softwaretechnik zur Koordination asynchroner Prozesse. Es dient der Entkopplung von Datenkomponenten, die Informationen erzeugen (*Producer*), und Komponenten, die diese Daten verarbeiten (*Consumer*). Die Kommunikation erfolgt dabei über eine gemeinsame Warteschlange (*engl. Queue*).

Für einen fehlerfreien Ablauf in nebenläufigen/konkurrenten Systemen müssen folgende zwei zentrale Synchronisationsprobleme gelöst werden [22, 23]:

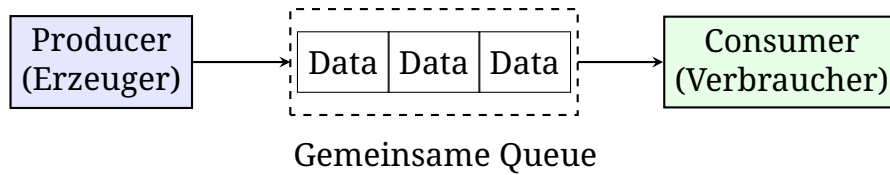


Abbildung 2.3: Architektur eines Producer-Consumer-Patterns (eigene Darstellung)

1. **Puffer-Überlauf:** Ist der Puffer vollständig gefüllt, muss der Producer so lange blockiert werden, bis der Consumer ein Element entnommen hat.
2. **Puffer-Leerlauf:** Ist der Puffer leer, muss der Consumer blockieren, bis der Producer neue Daten bereitstellt.

2.3.2 Task Parallel Library (TPL)

Die *Task Parallel Library (TPL)* ist eine .NET-Bibliothek, die es dem Programmierer vereinfacht, Parallelität und Nebenläufigkeit in C#-Programmen umzusetzen [24]. Für diese Arbeit ist vor allem das Konzept der Nebenläufigkeit (*engl.: concurrency*) relevant. Darunter versteht man allgemein, dass "mehr als nur eine Sache zur gleichen Zeit gemacht wird" [25, Kapitel 1]. In der praktischen Anwendung ermöglicht das eine Entkopplung von Rechenoperationen oder Ein-/Ausgabeoperationen vom primären Arbeitsstrang einer Anwendung. In Abbildung 2.4 werden synchrone und asynchrone

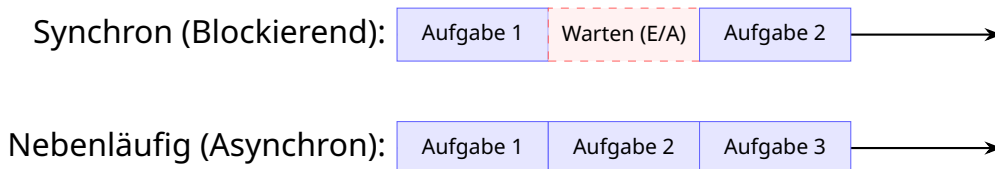


Abbildung 2.4: Problematik von blockierenden Abläufen (eigene Darstellung)

Abläufe gegenübergestellt. Es lässt sich erkennen, dass bei einem synchronen Ablauf eine Wartezeit den aufrufenden Thread blockiert - TPL löst diese Problematik durch die Einführung von asynchronen *Tasks*.

Nebenläufigkeit vs. Parallelität

Um die Rolle der TPL im Kontext dieser Arbeit zu präzisieren, muss zwischen Nebenläufigkeit und Parallelität unterschieden

werden. Während Parallelität die gleichzeitige Ausführung mehrerer Aufgaben auf verschiedenen Prozessorkernen beschreibt, definiert Nebenläufigkeit die organisatorische Strukturierung eines Programms. Ein nebenläufiges System ist darauf ausgelegt, Aufgaben durch verschachtelten Fortschritt (*engl.: Interleaving*) so zu bearbeiten, dass eine Anwendung reaktionsfähig bleibt, während sie im Hintergrund andere Prozesse abarbeitet.

Dabei ist Multithreading – also die Verwendung mehrerer Ausführungsstränge – lediglich eine Form der nebenläufigen Programmierung. Moderne Frameworks wie die TPL bieten hierfür übergeordnete Abstraktionen, die den direkten, fehleranfälligen Umgang mit Threads durch effizientere Konzepte ersetzen.

Task-Modell

Die TPL löst die starre Bindung an Betriebssystem-Threads durch das Konzept des Tasks (.NET-Bibliothek `System.Threading.Tasks`) auf. Ein Task repräsentiert eine asynchrone Operation und verkörpert das Entwurfsmuster eines Futures oder Promises – ein logisches Versprechen auf ein in der Zukunft liegendes Ergebnis. Anstatt Aufgaben fest an schwerfällige Betriebssystem-Threads zu binden, entkoppelt die TPL die logische Aufgabe von der physikalischen Ausführung. Ein interner Thread-Pool verwaltet eine minimale Anzahl von Arbeiter-Threads und verteilt die anfallenden Tasks dynamisch. Wartet ein Task beispielsweise auf eine Netzwerkanfrage, gibt er den physischen Thread sofort für andere nebenläufige Aufgaben frei.

2.3.2.1 Task-based Asynchronous Pattern (TAP)

Das Task-based Asynchronous Pattern (TAP) setzt das zuvor beschriebene Task-Modell ab .NET-Framework 4 als Standard für die asynchrone Programmierung in C# durch [23]. Hierzu kommen die Schlüsselwörter **async** und **await** zum Einsatz. Der Kern dieses Entwurfsmusters basiert auf den Abstraktionsebenen *Task* und *Task<TResult>*, welche asynchrone Operationen als eigenständige Funktionseinheiten im verwalteten (*engl. managed memory*) Speicher kapseln.

Die Deklaration einer Methode mit dem Schlüsselwort *async* signalisiert dem Compiler, dass innerhalb dieser Routine asynchrone Wartepunkte existieren können. Trifft der Kontrollfluss hingegen auf das Schlüsselwort *await*, wird die Ausführung der aktuellen Methode unterbrochen. Anstatt jedoch den aktuellen Thread (z.B. den Haupt- oder GUI-Thread) blockierend zu blockieren, wird der

Ausführungskontext eingefroren und die Kontrolle an den aufrufenden Prozess zurückgegeben.

Eine genauere Ausführung zu dieser Thematik ist unter [26] zu finden.

2.3.2.2 Task Cancellation/kooperativer Abbruch

Ein Problem asynchroner Hintergrundprozesse ist deren kontrollierte Beendigung. Da das gewaltsame Abbrechen von Threads (*engl. Thread Abortion*) aufgrund von potenziellen Speicherlecks und inkonsistenten Objektzuständen in modernen Laufzeitumgebungen als unzulässig gilt, implementiert das .NET-Ökosystem das Prinzip des *kooperativen Abbruchs*. Dieses beruht auf der Interaktion zweier Kernkomponenten:

- **CancellationTokenSource (CTS):** Diese Komponente fungiert als zentraler Signalgeber. Sie verwaltet den Abbruchzustand und löst bei Bedarf die Stornierung aus.
- **CancellationToken:** Dies ist ein Struktur-Token, welches von der CTS erzeugt und als Argument an asynchrone Tasks übergeben wird.

Der Abbruch erfolgt rein freiwillig und kooperativ. Der asynchrone Hintergrundprozess muss in regelmäßigen Abständen den Zustand des Tokens über die Eigenschaft *IsCancellationRequested* aktiv prüfen oder das Token an blockierende asynchrone Methoden weiterreichen.

2.4 Inverse Kinematik des Mitsubishi Movemaster EX RV-M1

Die inverse Kinematik, auch Rückwärtstransformation genannt, befasst sich mit der Bestimmung der Gelenkwinkel $\theta_1, \dots, \theta_i$ eines Roboters um eine vorgegebene Position und Orientierung des TCP in kartesischen Koordinaten zu erreichen. Der Index i steht dabei für den Freiheitsgrad des Roboters (*Degree's of Freedom*). Für den vorliegenden Mitsubishi Movemaster EX RV-M1 werden damit die fünf Gelenkwinkel analytisch über einen geometrisch-trigonometrischen Ansatz gelöst [1].

2.4.1 Kinematische Parameter des RV-M1

Für die Berechnung der Gelenkwinkel werden die Dimensionen der einzelnen Glieder benötigt. Diese sind nachfolgend in Tabelle 2.5 dargestellt: Die nachfolgenden Gleichungen basieren allesamt auf [1]. Ein

Achse	Länge [mm]
d_1	300
a_1	250
a_2	160
d_G	179

Abbildung 2.5: Kinematische Parameter des RV-M1

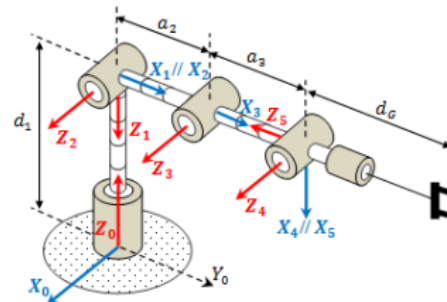


Abbildung 2.6: Skizze Denavit-Hartenberg-Frames des RV-M1 [1]

alternativer, auf diesen Gleichungen aufbauender bzw. herleitbarer Ansatz wurde zusätzlich in MATLAB implementiert. Das zugehörige Skript ist im digitalen Anhang ersichtlich.

2.4.2 Trigonometrische Kompaktnotation nach [1]

Zur Reduzierung der mathematischen Komplexität und für eine übersichtliche Darstellung der Transformationsmatrizen wird nach [1] eine standardisierte trigonometrische Kurzschreibweise eingeführt.

Dabei repräsentieren die Buchstaben c und s die Kosinus- bzw. Sinusfunktionen. Ein einzelner numerischer Index verweist auf das jeweilige isolierte Gelenk, während verkettete Indizes die Summe der entsprechenden Gelenkwinkel darstellen:

$$c_i = \cos(\theta_i) \quad (2.7)$$

$$s_i = \sin(\theta_i) \quad (2.8)$$

$$c_{ij} = \cos(\theta_i + \theta_j) \quad (2.9)$$

$$s_{ij} = \sin(\theta_i + \theta_j) \quad (2.10)$$

$$c_{ijk} = \cos(\theta_i + \theta_j + \theta_k) \quad (2.11)$$

$$s_{ijk} = \sin(\theta_i + \theta_j + \theta_k) \quad (2.12)$$

Diese Notation findet nachfolgend Verwendung für die Rückwärts-transformationsgleichungen.

2.4.3 Analytische Gelenkwinkelberechnung

Berechnen des Basiswinkels θ_1

Betrachtet man Abbildung 2.6, so lässt sich schnell erkennen, dass der Basiswinkel θ_1 unabhängig von den nachfolgenden Gelenken berechnet werden kann. Es reicht zu betrachten, wo die Zielkoordinaten liegen.

$$\theta_1 = \text{atan2}(X, Y) \quad (2.13)$$

Hier kommt die atan2-Funktion zum Einsatz mit welcher die Vorzeichen der Quadranten beachtet werden und eine eindeutige Winkelbestimmung im Bereich von -180° bis $+180^\circ$ möglich ist.

Berechnung Handwurzelpunkt

Um die Position des Handwurzelpunktes (*Wrist Center*) $[x_w, y_w, z_w]^T$ im Referenzkoordinatensystem zu bestimmen, wird die Transformation des Endeffektors (Greiferlänge d_G , 2.6) in Abhängigkeit des Neigungswinkels (Pitch) von den Zielkoordinaten subtrahiert. Zur Vereinfachung der Notation werden die trigonometrischen Funktionen im Folgenden mit $c_1 = \cos(\theta_1)$ und $s_1 = \sin(\theta_1)$ abgekürzt:

$$x_w = x_G - d_G \cdot t_{13} = c_1(a_2c_2 + a_3c_{23}) \quad (2.14)$$

$$y_w = y_G - d_G \cdot t_{23} = -s_1(a_2c_2 + a_3c_{23}) \quad (2.15)$$

$$z_w = z_G - d_G \cdot t_{33} = d_1 + a_2s_2 + a_3s_{23} \quad (2.16)$$

Berechnung des Schulterwinkels (θ_2)

Zur analytischen Bestimmung von θ_2 werden die Gleichungen der Handwurzelkoordinaten quadriert und addiert. Dies führt auf eine rekurrenente kinematische Gleichung der Form:

$$\alpha \sin \theta_2 + \beta \cos \theta_2 = \gamma \quad (2.17)$$

Die mathematischen Hilfsvariablen α, β, γ sowie der Radius r sind dabei wie folgt definiert:

$$\begin{cases} \alpha = z_w - d_1 \\ \beta = x_w c_1 - y_w s_1 \\ \gamma = \frac{a_2^2 - d_3^2 - \alpha^2 - \beta^2}{2a_2} \\ r = \sqrt{\alpha^2 + \beta^2} \end{cases} \quad (2.18)$$

Die geschlossene Lösung für das Schultergelenk lautet folglich:

$$\theta_2 = \text{atan2}\left(\gamma, \pm\sqrt{r^2 - \gamma^2}\right) - \text{atan2}(\alpha, \beta) \quad (2.19)$$

Eine Arbeitsraumprüfung erfolgt implizit über den Radikanden: Ist $r^2 - \gamma^2 < 0$, ist der Zielpunkt mathematisch imaginär und liegt außerhalb der physikalischen Reichweite des Roboters, was zum Abbruch des Berechnungsalgorithmus führen muss.

Berechnung des Ellbogenwinkels (θ_3)

Nachdem θ_2 bestimmt wurde, lässt sich der Ellbogenwinkel durch die Division der entkoppelten Positionsgleichungen auflösen:

$$\theta_3 = \text{atan2}(\beta - a_2 c_2, \alpha - a_2 s_2) - \theta_2 \quad (2.20)$$

Orientierungswinkel (θ_4, θ_5)

Um die Orientierung des Greifers im Raum unabhängig von der Armkonfiguration konstant zu halten, wird der absolute Neigungswinkel φ zur Horizontalen (Bodenreferenz) definiert:

$$\varphi = \theta_2 + \theta_3 + \theta_4 \quad (2.21)$$

Hieraus isoliert sich das Pitch-Gelenk θ_4 direkt über die Differenz:

$$\theta_4 = \varphi - \theta_2 - \theta_3 \quad (2.22)$$

Der Roll-Winkel des Handgelenks resultiert aufgrund der kinematischen Struktur linear aus der geforderten Eigenrotation des Endeffektors:

$$\theta_5 = \text{Roll} \quad (2.23)$$

3 Ausgangssituation/IST-Analyse

3.1 Architekturlimitationen des bisherigen Systems

3.1.1 Verwaltung der Punktwolken-Daten

Bei der Entwicklung der grafischen Benutzeroberfläche zeigte sich in der Ausgangslage ein kritisches Störverhalten innerhalb der Rendering-Engine des *PCLVisualizer*. Beim Versuch, im Viewer platzierte 3D-Objekte sequenziell über den Index *this->objectCount* zu entfernen, lief bereits der erste Löschvorgang fehlerhaft ab. Bei einem wiederholten Aufruf stürzte das Programm schließlich vollständig ab. Als Ursache hierfür wird vermutet, dass die zugrundeliegende Engine die Pointer-Referenzen oder die expliziten Speicheradressen der Objekte verliert.

Die anschließende Ursachenanalyse deutet auf ein sogenanntes Race-Condition-Problem hin, das durch ungeschützte, geteilte Ressourcen entsteht. Da sowohl die 3D-Visualisierung der Point Cloud Library (PCL) als auch die algorithmische Punkteverarbeitung zeitgleich auf dieselben Speicherbereiche zugreifen, kommt es hier zu kritischen Datenkonflikten.

Lösungsansätze und Gegenmaßnahmen:

Ein theoretischer Ansatz zur Stabilisierung ist die Absicherung der kritischen Speicherbereiche und der Visualisierungsinstanz mittels *std::mutex*, um threadübergreifende Zugriffe sauber zu blockieren. Aus Zeitmangel wurde diese Methode im realen Betrieb jedoch noch nicht getestet. Stattdessen wurde eine algorithmische Vereinfachung erfolgreich durchgeführt: Die fehleranfällige Einzellöschung über *removeShape* wurde durch die native PCL-Funktion *removeAllShapes* ersetzt. Diese Methode leert den gesamten Viewport in einem einzigen Schritt, was die Rendering-Engine entlastet. In den Praxistests funktionierte dieser Ansatz einwandfrei und behob die Abstürze vollständig.

Implementierung einer komfortablen Viewport-Konfiguration:

In der Vorgänger-Arbeit war zwar bereits die logische Auswahl vorgesehen, die Punktwolken vor jedem Neustart entweder komplett zu löschen oder zwei Punktwolken parallel anzuzeigen, allerdings war diese Konfiguration tief im Quellcode versteckt. Da dies bei der Bedienung schnell zu Fehlern führen konnte, wurde die Software um einen dedizierten Auswahlbutton direkt in der GUI erweitert. Der Benutzer kann die gewünschte Ansicht nun flexibel und sicher per Mausklick steuern. Der zugehörige, modifizierte Programmcode ist im Anhang dieser Arbeit dokumentiert.

3.1.2 Sequenzielle Befehlsstruktur

Die vorliegende Befehlsübermittlung an den Movemaster EX RV-M1 wird in der Software *Objekterkennung_RoboETH* sequenziell realisiert. 3.1 zeigt qualitativ auf, wie eine solche Abfolge aussieht. Es ist

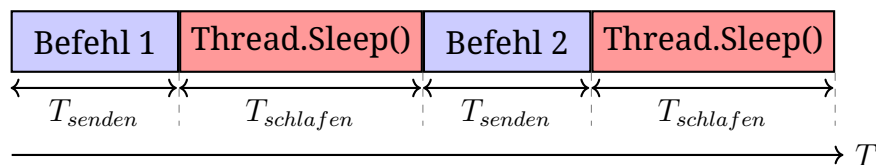


Abbildung 3.1: Sequenzielle Abfolge an Befehlen (Eigene Darstellung)

erkennbar, dass nach jedem gesendeten Befehl ein *Thread.Sleep()-Befehl* den gesamten Hauptthread für eine nicht vernachlässigbare Zeitspanne T_{schlafen} blockiert. Die theoretische Zeit T die benötigt wird, bis ein Befehl vollständig an den Roboter gesendet, von ihm verarbeitet und ausgeführt wird, lässt sich vereinfacht mit $T_{\text{befehlsausfuehrung}} = T_{\text{senden}} + T_{\text{schlafen}}$ beschreiben.

Während dieser Zeitspanne friert die gesamte Benutzeroberfläche sowie die Handlungsfähigkeit des Hauptthreads ein. Um die Auswirkungen der sequenziellen Befehlsbereitstellung auf die Gesamtlaufzeit und die Systemstabilität zu quantifizieren, ist eine detaillierte Analyse der Ausführungszeiten erforderlich. Hierfür wird in 3.1.2.2 ein reproduzierbares Messexperiment aufgesetzt.

Das Ziel dieser Analyse ist die Identifikation der kritischen Pfade. Durch eine explizite Messung der Zeitstempel für den Eintritt in die Befehlsverarbeitung, die Dauer der Hardware-Kommunikation und die Verweildauer in den Wartezuständen wird der „Flaschenhals“ der synchronen Architektur isoliert. Erst durch diese messtechnische

Herleitung wird transparent, an welcher Stelle die CPU-Last ungenutzt bleibt und wie sich die Thread-Auslastung sowie die Laufzeit bei einer Umstellung auf eine asynchrone Pipeline-Architektur verbessern lässt.

3.1.2.1 Theoretische Limitation

Wird erneut Abbildung 3.1 betrachtet, so fällt auf, dass die Zeit, welche benötigt wird, um die Koordinaten der einzelnen Objekte im Arbeitsbereich des Roboters zu berechnen, nicht eingerechnet wird. Grund hierfür ist die Annahme, dass die Berechnungszeit in beiden Fällen (sequenzielle Befehlsstruktur oder asynchrone Befehlspipeline) voraussichtlich gleich ist und im Vergleich mit den Ausführungszeiten der physischen Roboterbewegung vernachlässigbar ist.

Die theoretische Gesamtlaufzeit T_s einer Befehlssequenz lässt sich somit durch Summation der Befehlsausführungszeiten $T_{befehlsausfuehrung}$ bestimmen:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k T_{befehlsausfuehrung,i,j} \quad (3.1)$$

Einsetzen der zuvor beschriebenen Zeiten ergibt:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.2)$$

Hierbei repräsentiert der Index i das jeweilige Objekt innerhalb der Bearbeitungsreihenfolge (mit n als Gesamtzahl der Objekte), während der Index j die spezifische Befehlsposition innerhalb eines Manipulationszyklus angibt (mit k als Anzahl der Befehle pro Objekt).

In den Gleichungen 3.1 und 3.2 lässt sich die direkte Korrelation zwischen dem Parameter n und der durch die Bildverarbeitungsklasse erfassten Objekte erkennen. Des Weiteren wird verdeutlicht, dass die Gesamtausführungszeit einer Befehlssequenz (bestehend aus $1, \dots, k$ Befehlen pro Objekt) linear von der Objektanzahl n und der aufsummierten Wartezeiten durch *Thread.Sleep()*-Befehlen abhängt.

Bei dieser theoretischen Betrachtung wird bewusst eine Worst-Case-Abschätzung mittels der fest definierten Wartezeiten getroffen, um sicherzustellen, dass der Roboter die Bewegung garantiert abgeschlossen hat, bevor der nächste Befehl gesendet wird. Dabei werden die Wartezeiten entweder auf Basis von experimenteller

Erfahrung oder durch grobe Berechnung der benötigten Zeit gesetzt. Praktisch funktioniert diese Vorgehensweise, allerdings können hieraus Einbußen in der Systemeffizienz und -dynamik resultieren.

3.1.2.2 Messexperiment

Um die theoretisch hergeleiteten Ineffizienzen und die zeitlichen Auswirkungen der sequenziellen Befehlserteilung quantitativ nachzuweisen, wurde ein deterministisches Messexperiment aufgesetzt. Selbiges muss für Vergleichbarkeit und Bewertbarkeit auch bei nachfolgenden Erweiterungen des Systems (siehe 4.2) zu Grunde gelegt werden.

Das Messexperiment wird mit vier Zylinderobjekten durchgeführt - die Positionen sind jeweils auf der Arbeitsunterlage markiert. Abbildung 3.2 zeigt die Anordnung der Objekte sowie die zugehörigen Objektkoordinaten. Die Geschwindigkeit des Roboters wird per Befehl



(a) Objektanordnung

Objekt-ID: X-, Y-, Z-, Roll, Pitch

Objekt 1: -42, 291, 6, -90, -8

Objekt 2: -37, 374, 6, -90, -5

Objekt 3: 36, 295, 6, -90, 6

Objekt 4: 55, 346, 6, -90, 9

(b) Koordinaten im Messexperiment

Abbildung 3.2: Messexperiment zur Evaluation der Sortierperformance (eigene Darstellung)

SP fest auf Stufe 5 H (0-9 Stufen, H = Hohe Beschleunigung) gesetzt. Sowohl die maximalen als auch die eingestellten Geschwindigkeiten der einzelnen Achsen lassen sich aus Tabelle 3.1 entnehmen und sind für die spätere Betrachtung der inversen Kinematik des Movemaster EX RV-M1 relevant.

Achse	maximale Geschwindigkeit SP9	SP5-Geschwindigkeit $\dot{\theta}_{\max,i}$
Joint 1	120 °/sec	29,64 °/sec
Joint 2	72 °/sec	17,78 °/sec
Joint 3	109 °/sec	26,92 °/sec
Joint 4	100 °/sec	24,7 °/sec
Joint 5	163 °/sec	40,26 °/sec

Tabelle 3.1: maximale bzw. SF5-Gelenkgeschwindigkeiten nach [27, APP-52, 1-7]

Das Ziel, alle Objekte jeweils in einen leeren Magazinplatz zu sortieren, kann in vier Objektsortierungssequenzen und zwei Sequenzen für die Start-/Endposition eingeteilt werden. Dabei enthält jede Objektsortierungssequenz Fahr- und Greifbefehle - ersichtlich in Abbildung 3.3.

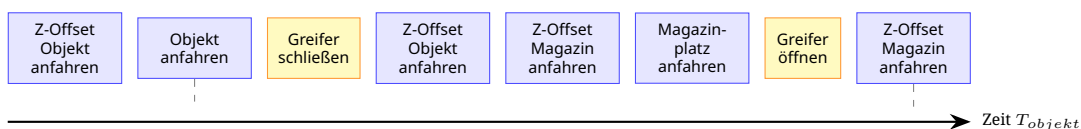


Abbildung 3.3: Befehle innerhalb einer Objektsortierungssequenz (eigene Darstellung)

3.1.2.3 Praktische Limitation - Laufzeitmessung mit Simulationsexperiment

Um die praktische Limitation der bisherigen Architektur sinnvoll beurteilen zu können, wird zunächst die theoretisch benötigte Laufzeit mit Gleichung 3.2 ermittelt.

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{\text{senden},i,j} + T_{\text{schlafen},i,j})$$

Betrachtet man den Programmcode der Klasse *Roboter.cs*, Methode *SendData*, so fällt auf, dass jeder Sendevorgang ($T_{\text{senden},i,j}$) mit einem `Thread.Sleep()` von 800ms beaufschlagt wird. Dies geschieht, um sicherzugehen, dass innerhalb der Befehlsweiterleitung von C#-Programm zu RoboETH und letztlich über RS232C an den Movemaster EX RV-M1 genügend Zeit für die RoboETH-Bridge vorhanden ist, um die Befehle zu verarbeiten und an den Roboter weiterzuleiten. Das verhindert Kommandoüberläufe sowie einen Absturz der Bridge und ermöglicht

ein sicheres Empfangen über RS232C.

Um die festgelegte Zeit $T_{schlafen,i,j}$ für ein Objekt zu ermitteln, werden die Methoden *SendKoordinations*, *GoToMagazin* sowie *GoToStartPos* betrachtet und in der nachfolgenden Tabelle dargestellt. Setzt man nun die statischen Werte aus Tabelle 3.2, die Wartezeiten bei

Beschreibung	Thread.Sleep() [ms]	$T_{schlafen,i,j}$
Fahrt auf Objekt Z-Offset	3000	$T_{schlafen,i,1}$
Absenken auf Objektposition	3000	$T_{schlafen,i,2}$
Objekt greifen	500	$T_{schlafen,i,3}$
Anheben auf Objekt Z-Offset	2000	$T_{schlafen,i,4}$
Fahren auf Magazin Z-Offset	5000	$T_{schlafen,i,5}$
Absenken auf Magazinposition	1000	$T_{schlafen,i,6}$
Loslassen des Objekts	1000	$T_{schlafen,i,7}$
Anheben auf Magazin Z-Offset	1000	$T_{schlafen,i,8}$
<i>Anheben auf Magazin Z-Offset</i>	1000	$T_{schlafen,i,9}$

Tabelle 3.2: Tabellarische Übersicht der Thread.Sleep()-Verzögerung pro Objektsequenz

jedem Senden sowie die benötigte Zeit für das Erreichen von Start- und Endposition in die Gleichung für die Gesamtdauer T_S ein

$$T_S = T_s = 2 * (1800ms) + \sum_{i=1}^4 \sum_{j=1}^9 (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.3)$$

so erhält man eine berechnete Gesamtdauer von 102400 ms. Im Rahmen der Ist-Analyse der vorliegenden Architektur wurde mit der *Stopwatch*-Klasse im C#-Namensraum *System.Diagnostics* präzise gemessen, wie lange die gesamte softwareseitige Ausführung der Sortieraufgabe benötigt. Dies ist nachfolgend in 3.3 dargestellt.

Beim Vergleich der berechneten und gemessenen Laufzeiten lässt sich erkennen, dass die aktuelle Architektur hinsichtlich des *Overheads* bereits sehr performant ist (Abweichung von 1,3% zwischen Berechnung und Messung).

Beurteilung des IST-Standes

Während der Durchführung des Messexperiments am aktuellen Softwarestand ließ sich erkennen, dass die Hardware-Anforderungen des

Objekt-ID	Theoretische Zeit T_S [s]	Gemessene Zeit T_{mess} [s]
Startposition	1,8	2,01
Objekt 1	24,7	24,97
Objekt 2	24,7	24,91
Objekt 3	24,7	24,95
Objekt 4	24,7	24,89
Endposition	1,8	2,00
T_S	102,40	103,73

Tabelle 3.3: Vergleich der theoretischen Laufzeit T_S mit den gemessenen Werten T_{mess}

Movemaster EX RV-M1 souverän gemeistert werden und der Sortierungsablauf zuverlässig erfolgt. Allerdings konnten visuell Stillstandszeiten ausgemacht werden, welche die Dynamik des Roboters vor allem bei folgenden Bewegungen stark einschränken:

- Absenken und Anheben des Roboters auf Z-Offset (über Objekt oder Magazin)
- Objektfahrten
- Magazinfahrten

Die Abschätzung hinsichtlich der benötigten Zeiten für das Greifen/Loslassen eines Objekts sowie die Fahrt zu Start-/Endposition wurde hingegen gut getroffen. Auf Basis dieser Erkenntnisse wird in Kapitel 4 ein SOLL-Stand der Architektur erarbeitet und aufgezeigt, wie eine mögliche Implementierung aussehen kann.

4 Architekturoptimierung und -erweiterung

4.1 Konzeptionierung der dynamischen Robotersteuerung

In diesem Kapitel werden Möglichkeiten erläutert, wie die Objektsortierung durch den Movemaster EX RV-M1 beschleunigt, bzw. in erster Linie dynamisiert werden kann. Unter Dynamisierung wird nachfolgend das weitflächige Eliminieren von blockierenden Thread.Sleep()-Methoden eingeordnet.

4.1.1 Optionen zur Positionserkennung/Befehls-Bewegungssynchronisation

Um eine dynamische und nahezu unterbrechungsfreie Befehlserteilung zu realisieren, ist ein Mechanismus unerlässlich, der der Steuerungssoftware mitteilt, wenn die physische Bewegung des Roboters vollständig abgeschlossen ist. Nachfolgend werden unterschiedliche Möglichkeiten betrachtet, mit denen ein solches *Acknowledgement* erzielt werden kann.

Ereignisbasiertes Hardware-Feedback

Ein übliches Verfahren zur Synchronisation des Zustandes der Steuerungssoftware mit der aktuellen Position des Roboters stellen ereignisbasierte Architekturen (event-driven architectures) dar. Hierbei sendet der Roboter bei Erreichen der Zielposition und -orientierung ein Quittierungssignal über den Kommunikationskanal zurück an die Steuerung. Innerhalb der Steuerung wird ein *Event-Handler* aktiviert, der den weiteren Ablauf einleitet. Allgemeine Vorteile eines solchen Systems sind nahezu sofortige Reaktionszeiten der Steuerung sowie niedrige Datenmengen [28].

Der Mitsubishi Movemaster EX RV-M1 bietet allerdings keine hardwareseitige Implementation einer solchen Rückmeldung. Die Drive Unit des Roboters quittiert den fehlerfreien Erhalt eines per RS232C gesendeten Befehl, liefert allerdings keine Auskunft über

den Abschluss der Roboterbewegung [27, Vgl.]. Eine ereignisbasierte Architektur über den Standard-Befehlssatz und die Standard-Hardware des EX RV-M1 ist somit ausgeschlossen.

Zyklische Zustandsabfrage mit 'WH'-Befehl

Eine alternative Variante zur Bestimmung des aktuellen Roboterzustands stellt ein zyklisches Abfragen (*engl. Active Polling*) der Gelenkwinkel oder der Position in Weltkoordinaten dar. Dies kann über den Befehl *WH* (Where), welcher von der Steuerungssoftware kontinuierlich an den Roboter gesendet wird, realisiert werden.

Innerhalb der Steuerungssoftware muss eine Methode implementiert werden, die die zurückgemeldeten Ist-Koordinaten mit den Zielkoordinaten vergleicht.

$$\|X_{\text{Ist}} - X_{\text{Soll}}\| < \epsilon \quad (4.1)$$

Befindet sich der Roboter innerhalb eines definierten Toleranzrahmens ϵ , so kann der nächste Befehl freigegeben werden.

Für diese Studienarbeit wurde der Ansatz experimentell evaluiert und aufgrund von Systemlimitationen als unpraktikabel und unzureichend für eine dynamische Steuerung eingestuft. Grund hierfür sind folgende Probleme:

- **Schnittstellen-Überforderung:** Bei dem Versuch, während der Ausführung von Bewegungen per *WH* die Position des Roboters zu erfragen, fiel auf, dass dies zuverlässig zu einem Fehlerzustand in der Drive Unit führt. Auch die Applikation *RoboETH* stürzt ab.
- **Latenz:** Eine kontinuierliche Abfrage der Position durch das Senden von *WH*-Befehlen erzeugt ein hochfrequentes *Request-Response-Rauschen* auf der RS232C-Schnittstelle. Das kann Auswirkungen auf andere zeitkritische Steuerbefehle (wie *MP*- und *GO*-/GC-Befehle) haben.

Deswegen wird auch dieser Ansatz für das weitere Vorgehen verworfen.

Prädiktive Zeitberechnung mit inverser Kinematik

Da eine direkte Rückmeldung des Roboters sowie ein aktives Abfragen der Position nicht realisierbar sind, wird in diesem Unterkapitel die Möglichkeit einer prädiktiven Zeitberechnung *engl. predictive time-scaling* mithilfe der inversen Kinematik vorgestellt. Durch die Bilderkennung sind bereits vor Beginn der Bewegungsausführung alle Koordinaten der Objekte sowie die fest definierten Koordinaten der Magazinpositionen bekannt. Mithilfe dieser werden über die in Kapitel 2.4 hergeleiteten Gleichungen die erfordernten Gelenkwinkel des Roboters zum Erreichen der gewünschten Position und Orientierung bestimmt.

Bei der Verwendung von *MP* führt der EX RV-M1 eine synchrone Point-To-Point-Bewegung aus. Darunter versteht man, dass die Bewegung aller fünf Achsen gleichzeitig gestartet und abgeschlossen wird (siehe Abbildung 4.1). Die Gesamtdauer der Bewegung wird

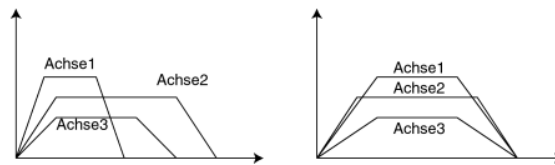


Abbildung 4.1: Bewegungsablauf asynchrone/synchrone PTP nach [29]

folglich durch die langsamste Achse vorgegeben - diese nennt man bei synchronen PTP-Bewegungen Leitachse (*engl. Leading Axis*) [29, 27]. Mit der inversen Kinematik werden für jeden Zielpunkt die Gelenkwinkeldifferenzen $\Delta\theta_i$ (mit $i = 1, \dots, 5$) berechnet. Bezieht man die maximalen Winkelgeschwindigkeiten $\dot{\theta}_{\max,i}$, erläutert in Tabelle 3.1 ein, so erhält man folgende Gleichung:

$$T_{\text{Fahrt}} = \max_{i=1}^5 \left(\frac{|\theta_{i,\text{Ziel}} - \theta_{i,\text{Start}}|}{\dot{\theta}_{\max,i}} \right) \quad (4.2)$$

Diese berechnete Zeit T_{Fahrt} bildet die fundamentale Zeitbasis für das nachfolgende Kapitel.

4.2 Befehlswarteschlange für dynamische Robotersteuerung

Um die theoretisch hergeleitete Zeitberechnung in der Steuerungssoftware umzusetzen, muss zunächst die gewünschte Funktionsweise definiert werden. Hierfür sind vor allem die in Kapitel 2.3.1 und 2.3.2.1 erläuterten Grundlagen relevant.

Der wesentliche Architekturwechsel von einem rein sequenziellen, blockierenden Steuerungsansatz hin zu einer entkoppelten, ereignisgesteuerten Pipeline ist in Abbildung 4.2 gegenübergestellt.

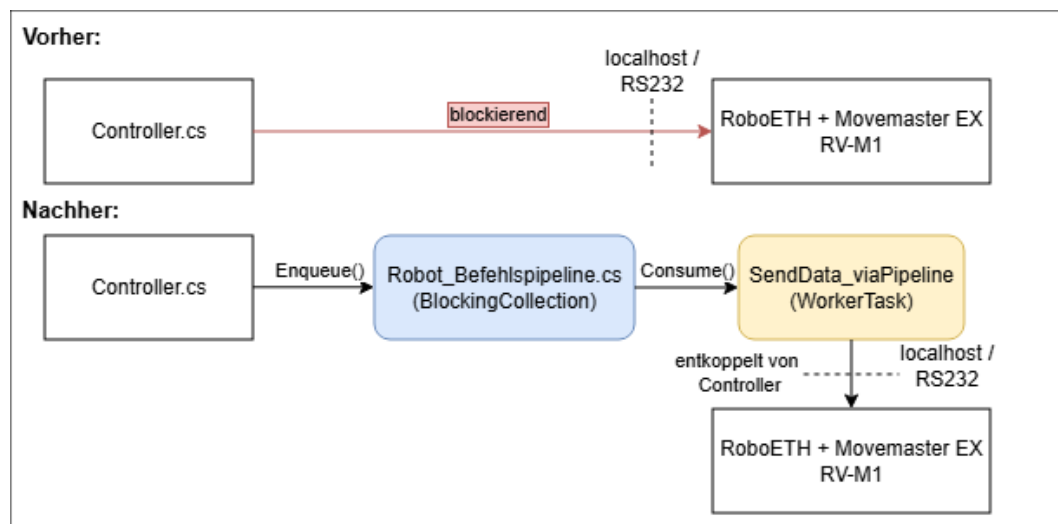


Abbildung 4.2: Vergleich zwischen monolithischer Befehlsverarbeitung und entkoppelter Pipeline-Architektur

Die softwaretechnische Umsetzung dieser Pipeline-Architektur setzt drei Kernaspekte der dynamischen Robotersteuerung um:

1. **Entkopplung während der Befehlsübertragung:** In der ursprünglichen Systemarchitektur blockierte jeder Sendevorgang den übergeordneten Steuerungs-Thread. Durch die Implementierung der *Robot_Befehlspipeline* agiert die Steuerungsklasse *Controller.cs* nun rein als *Producer*. Fahrbefehle werden als Datenpakete mittels der Methode *Enqueue()* in die *BlockingCollection* eingereiht. Dieser Vorgang ist nahezu latenzfrei, wodurch der Orchestrator unmittelbar für weitere Berechnungen, wie die Auswertung der Punktwolke oder GUI-Interaktionen, freigegeben wird.

2. **Asynchronität der Abarbeitung:** Die physische Befehlsausführung wird vollständig in einen dedizierten Hintergrund-Task (*Worker-Task*) verlagert, der nach dem *Task-based Asynchronous Pattern* (TAP) operiert. Der *Worker-Task* konsumiert die eintreffenden *RobotCommand*-Instanzen sequenziell.

Die Synchronisation zwischen der Software-Ebene und der physischen Hardwarebewegung erfolgt asynchron über *await Task.Delay()*. Dadurch wird verhindert, dass CPU-Ressourcen durch blockierende Warteschleifen (*Thread.Sleep*) verschwendet werden. Die Pipeline kontrolliert sich selbst: Ist die Queue leer, verbleibt der Hintergrund-Task in einem ressourcenschonenden Blockierungszustand, bis ein neues Element in die Queue geschickt wird.

3. **Dynamische Fahrtzeitberechnung:** Die größte Herausforderung einer entkoppelten Steuerung ohne reales Hardware-Feedback liegt darin, ein Voreilen der Software zu verhindern. Würden Befehle ohne abgestimmte zeitliche Basis an den Roboter gesendet, liefe der interne Puffer des seriellen Controllers über, was zu Datenverlust oder unkontrollierten Bewegungsabbrüchen führen würde.

Die Pipeline löst dieses Problem durch eine prädiktive Zeitsteuerung. Für jeden eingereichten Fahrbefehl wird im Vorfeld die Verfahrzeit der Leitachse basierend auf dem inversen kinematischen Modell 2.4 berechnet. Dieser Wert wird dem *RobotCommand* als Eigenschaft übergeben und zwingt den asynchronen *Consumer*, so lange mit dem nächsten Sendevorgang zu warten, bis der Roboter die Zielposition physikalisch erreicht hat.

5 Implementation der Erweiterungen

5.1 Safetyimplementation

5.1.1 Kriterienbasierte Evaluierung der Sicherheitsüberwachungssysteme

In der modernen industriellen Produktion hat sich eine Vorgehensweise etabliert, die auf einer systematischen Taxonomie der Aufgaben und Anwendungsfelder basiert. Dieser Ansatz ist maßgeblich, um funktionierende sowie finanziell und funktionell optimierte Absicherungssysteme zu entwickeln. Die Interaktion zwischen Mensch und Maschine wird in diesem Kontext standardmäßig als Human-Robot Interaction (HRI) beziehungsweise im industriellen Umfeld als Mensch-Roboter-Kollaboration (MRK) abgekürzt.

Die Auswahl einer geeigneten MRK-Sicherheitsüberwachung stützt sich auf diverse technologische und funktionale Entscheidungskriterien. Diese werden in der folgenden Evaluierungsmatrix anhand zentraler Kategorien – wie dem Aufbau des Arbeitsbereichs, dem den Notzustand einleitenden Ereignis, der eingesetzten Sensorik sowie der resultierenden Systemreaktion des Roboters – gegenübergestellt. Um den Rahmen dieser Arbeit zu wahren, konzentriert sich die Darstellung für jede Kategorie exemplarisch auf eine Gegenüberstellung zweier primärer Ausprägungen (Variante A und Variante B), wenngleich in der Praxis weitere technologische Mischformen existieren.

Kriterium	Variante A	Variante B
Sicherheitsbereich	statisch	dynamisch
Ereignis	Objekterkennung	Handerkennung
Sensorik	physisch	visuell
Aktion	Nothalt	Speed and Separation-Prinzip

Tabelle 5.1: Entscheidungsmatrix zur Auswahl der MRK-Sicherheitsüberwachung

Für die informationstechnische Abgrenzung des Kollaborationsraumes stehen sich statische und dynamische Konzepte gegenüber. Ein statischer Sicherheitsbereich zeichnet sich primär durch die Einfachheit seiner Umsetzung aus. Die Begrenzung kann entweder über einen physischen, kontrastreichen Rahmen im realen Hintergrund des Arbeitsbereichs oder rein softwareseitig über die Definition eines virtuellen Rahmens (Region of Interest, ROI) innerhalb der Bildmatrix realisiert werden. Eine Kaskadierung des Raumes in mehrere Teilzonen ist zudem möglich und stellt das grundlegende Fundament für die Implementierung des Speed and Separation Monitoring (SSM) dar. Die für diese Studienarbeit relevanteste Methode nutzt das definierte Blickfeld (Field of View, FoV) einer 2D-Kamera als feste Begrenzungsebene. Erkennt der Algorithmus innerhalb dieses Sichtfeldes eine Verletzung der definierten Grenzen durch eine Bedrohung, wird die Sicherheitsreaktion des Roboters initiiert. Da der Sichtbereich der Kamera hierbei als absolute Grenze fungiert, spielt die exakte geometrische Ausrichtung auf den zu betrachtenden Arbeitsraum eine prozesskritische Rolle.

Demgegenüber passt sich ein dynamischer Sicherheitsbereich adaptiv in Echtzeit an die aktuelle Pose des Roboterarms an. Die Komplexität dieser Implementierung ist bedeutend höher als bei statischen Systemen, da für die Hüllkurvenberechnung permanent die Vorwärtskinematik des Roboters gelöst und die Transformation der Koordinatensysteme in den dreidimensionalen Raum berechnet werden müssen. Dies bedingt eine sehr geringe Datenbus-Latenz (Echtzeitfähigkeit), stellt hohe Anforderungen an die Rechenleistung des Gesamtsystems und erfordert für den Werker eine dynamische, visuelle Anzeige des aktiven Bereichs, beispielsweise über Lichtprojektionen oder innerhalb eines Digitalen Zwillings. Der technologische Mehraufwand resultiert jedoch in einer maximalen Interaktionsfreiheit zwischen Mensch und Roboter und bietet eine hervorragende Kombinierbarkeit mit dem SSM-Prinzip.

Das Fraunhofer-Institut für Fabrikbetrieb und -automatisierung IFF hat genau eine solche Technologie zur optischen Arbeitsraumüberwachung patentieren lassen. Die Entwickler setzen auf einen lichtprojizierten Sicherheitsrahmen welcher von der Bildverarbeitung als Kontrast erkannt und bei Unterbrechungen ein Nothalt des Roboters veranlasst wird. Der auf den Hintergrund projizierte Rahmen bietet den Vorteil, dass sich seine Position parallel zum Roboter anpassen lässt. In Abbildung 5.1 ist eine visuelle Veranschaulichung des Prinzips geboten.

Bei der algorithmischen Ereignissteuerung wird zwischen universeller Objekterkennung und spezifischer Handerkennung unterschieden. Dient eine generelle Objekterkennung als auslösendes Ereignis für einen Sicherheitsstopp oder eine Geschwindigkeitsreduzierung, basiert dies meist auf klassischer Kantenextraktion oder



Abbildung 5.1: Projektions- und kamerabasierte Technologie zur Arbeitsraumüberwachung

Hintergrundsubtraktion. Sobald ein Fremdobjekt im Arbeitsbereich detektiert wird, hält die Anlage an. Ein inhärentes Problem dieses Ansatzes liegt darin, dass der Roboterarm selbst als Objekt erfasst wird und permanent aus dem Bild herauskalkuliert (maskiert) werden muss, was den Programmieraufwand wiederum erhöht. Der wesentliche Vorteil liegt in der einfachen grundlegenden Implementation. Als kritischer Nachteil erweist sich jedoch die hohe Fehlerwahrscheinlichkeit, da transiente Umgebungsstörungen fälschlicherweise als Pseudoobjekte erkannt werden. Diese hohe Rate an Falsch-Positiv-Detektionen führt zu häufigen unplanmäßigen Anlagenstillständen, was die wirtschaftliche Gesamtanlageneffizienz (Overall Equipment Effectiveness, OEE) massiv senkt.

Eine selektivere Alternative stellt die ereignisbasierte Handerkennung dar. Hierbei detektiert ein spezielles, auf die Erkennung menschlicher Hände trainiertes KI-Modell die Extremitäten des Werkers. Sobald eine Hand oder Teile davon im Sichtfeld erfasst werden, wechselt das System in den definierten Sicherheitszustand. Diese Methode ist aufwendiger zu programmieren und benötigt eine signifikant höhere Rechenleistung (Schnittstellen für GPU/NPU). Sie arbeitet jedoch im industriellen Umfeld wesentlich zuverlässiger und minimiert die Erkennung von Pseudoobjekten. Dennoch besteht das Restrisiko einer fehlerhaften oder verzögerten Klassifikation (Falsch-Negativ-Rate). Zudem muss die Auslegung gemäß DIN EN ISO 13855 zwingend von der maximalen menschlichen Handgeschwindigkeit von 1600 mm/s bis 2000 mm/s ausgehen. KI-Modelle müssen daher eine entsprechend hohe Bildwiederholrate (Frames per Second, fps) aufweisen, um diese Dynamik latenzfrei abzugreifen. Ein ungelöstes Problem bleibt die mathematische "Black-Box-Problematik" von Deep-Learning-Algorithmen, da diese schwer deterministisch zu

verifizieren sind. Für sicherheitskritische Systeme nach ISO 13849-1 ist der lückenlose Nachweis einer fehlerfreien Entscheidung eine der größten aktuellen Forschungsherausforderungen. Zudem zeigt sich eine strikte Abhängigkeit von den Trainingsdatensätzen; weisen diese keine hinreichende Varianz bezüglich Hautfarben, Arbeitshandschuhen oder Handhaltungen auf, droht ein potenzieller Ausfall der Schutzfunktion.

Auf der sensorischen Ebene stehen sich physische und visuelle Systeme gegenüber. Die physische Sensorik realisiert die Raum- und Kontaktherausforderung über klassische Komponenten wie Laserschranken um den Sicherheitsbereich, kapazitive Sensorfelder (z.B. als sensitive Roboterhaut) oder induktive Felder, die auf die Annäherung von Körperteilen reagieren. Physische Sensoren sind kostengünstig und zeichnen sich durch eine einfache Instandhaltung aus. Zudem eignen sie sich hervorragend als diversitäre, komplementäre Redundanz in Kombination mit optischen Systemen, um die strengen Anforderungen an die Ein-Fehler-Sicherheit nach DIN EN ISO 13849-1 [10] zu erfüllen. Als Nachteil erweist sich die anspruchsvolle Kalibrierung, da die Sensorik sensibel auf elektromagnetische Umweltstörungen (EMV) reagiert und oft spezialisierte Auswerteelektronik benötigt. Da physische Sensoren zudem keine 3D-Tiefen- oder Richtungsinformationen liefern, können sie weder die Annäherungsgeschwindigkeit noch die genaue Position im Raum bestimmen. Ein zusätzlicher Faktor ist der mechanische Verschleiß sowie der hohe Montage- und Verkabelungsaufwand im rauen Industriealltag. Die visuelle Sensorik nutzt Kamerasysteme (2D-Systeme mit Kantenextraktion oder 3D-Time-of-Flight-Sensoren mit Tiefendaten) zur volumetrischen Raumüberwachung. Visuelle Systeme bieten den Vorteil einer zuverlässigen Klassifizierung spezieller Geometrien und erlauben Mehrfachanwendungen, indem dieselbe Hardware sowohl für Sicherheitsaufgaben als auch für bildverarbeitungsgestützte Greifprozesse (Pick-and-Place) genutzt werden kann. Eine laufende Neukalibrierung im Betrieb ist meist nicht notwendig.

Demgegenüber stehen hohe Anschaffungskosten und eine aufwendigere algorithmische Programmierung. Zudem reagieren optische Systeme empfindlich auf wechselnde Beleuchtungsverhältnisse, Schattenwurf oder Verschmutzungen der Linse. Normativ ist hierbei die Zertifizierung nach IEC 61496 zu beachten: Optische Kamerasysteme für Sicherheitsanwendungen müssen zwingend als berührungslos wirkende Schutzeinrichtungen (ESPE) nach IEC 61496-1 zertifiziert sein. Standardmäßige Industriekameras ohne inhärente Hardware-Redundanz und gegenseitige Prozessorüberwachung sind

für sicherheitsgerichtete Abschaltungen unzulässig.

Als finale Aktuatorik stehen der abrupte Nothalt und das adaptive Abbremsen zur Auswahl. Ein Nothalt führt bei einer Schutzfeldverletzung zum sofortigen Stillsetzen des Roboterarms, geregelt über eine kontrollierte oder unkontrollierte Energietrennung (Kategorie 0/1 nach DIN EN 60204-1). Im Sicherheitskontext von Robotersystemen wird diese Reaktion gemäß ISO 10218-1 präzise als sicherheitsgerichteter überwachter Halt (Safety-rated monitored stop) definiert. Sie bietet die höchste Sicherheitsstufe zur Risikominimierung. Für kooperative Arbeitsräume ist diese Reaktion jedoch ineffizient, da jede Annäherung des Werkers zu einem vollständigen Produktionstopp führt. Die abrupten Momentenmaxima verursachen zudem einen massiven mechanischen Verschleiß der Robotergetriebe und Gelenke. Darüber hinaus reduziert die erhebliche Wiederanlaufzeit des Gesamtsystems (Bremsen lösen, Synchronisation der Servoregler) die Produktivität der Anlage signifikant. Eine wesentlich ergonomischere Lösung für die MRK stellt das Abbremsen des Roboterarms im Sinne des Speed and Separation Monitoring (SSM) dar. Der Manipulator reduziert seine Bahngeschwindigkeit in Abhängigkeit vom Abstand zum Menschen stufenweise oder kontinuierlich über vordefinierte Geschwindigkeitszonen. Diese Funktion der sicheren Geschwindigkeitsreduzierung (Safely-Limited Speed, SLS) sichert fließende Prozesse, minimiert den mechanischen Verschleiß und muss über sichere Antriebsregler (z.B. gemäß IEC 61800-5-2) validiert werden, um die funktionale Sicherheit zu garantieren.

Der Ansatz erfordert jedoch einen hohen softwareseitigen Aufwand für die Bildverarbeitung und verlangt bei Fehlfunktionen in der Regelung eine konsequente zweikanalige Überwachung. Zu beachten bleibt das regelungstechnische Restrisiko durch Trägheit: Aufgrund der Eigenmasse des Manipulators besitzt dieser auch während des Verzögerungsvorgangs eine hohe kinetische Energie. Das Sicherheitskonzept muss mathematisch garantieren, dass die Drosselung schnell genug erfolgt, bevor der Mensch die nächste kritische Distanzschwelle erreicht. Da der Roboter auch bei geringen Geschwindigkeiten hohe statische Kräfte aufbauen kann, muss das System so ausgelegt sein, dass ein Quetschen von Körperteilen zwischen dem Roboter und starren Umgebungsstrukturen physikalisch oder algorithmisch vollständig ausgeschlossen ist.

5.1.2 Machine-learning Modell zur visuellen Handerkennung

5.1.2.1 Machine learning-Modell

Die vorangegangenen Betrachtungen der unterschiedlichen sensorischen und algorithmischen Ansätze verdeutlichen, dass jede der klassischen Methoden in der industriellen Praxis an systemspezifische, physikalische Grenzen stößt. Während die Infrarot- und Wärmebildsensorik eine hohe Anfälligkeit für thermische Störgrößen durch umliegende Maschinenteile aufweist, schränken die normative Komplexität und die Latenzen reiner Time-of-Flight-Scanner den fließenden kollaborativen Prozess ein. Die klassische, rein farbmetrische Hautfarbenerkennung wiederum versagt beim Einsatz vorgeschriebener persönlicher Schutzausrüstung wie Arbeitshandschuhen und reagiert hochgradig sensibel auf transiente Beleuchtungswechsel im Fabrikumfeld.

Ein Blick auf aktuelle, umfassende Meta-Studien zum Stand der Technik verdeutlicht, dass man in der modernen Sicherheits- und Automatisierungstechnik heute primär auf Ansätze trifft, die auf Verfahren der Künstlichen Intelligenz beziehungsweise des Maschinellen Lernens basieren [16]. Innerhalb dieses datengetriebenen Leitbildes existiert eine Vielzahl unterschiedlicher Algorithmen und mathematischer Modelle, die sich in ihrer Komplexität und ihren Hardware-Anforderungen stark voneinander unterscheiden.

Für den methodischen Rahmen der vorliegenden Arbeit ist dabei insbesondere das Prinzip des Kaskadenklassifikators (*Cascade Classifier*) von Bedeutung, welcher im Kern auf dem klassischen Viola-Jones-Algorithmus basiert. Obwohl dieses Verfahren im direkten Vergleich zu hochkomplexen, modernen Deep-Learning-Modellen wie beispielsweise YOLOv8n (*You Only Look Once*) zu den älteren und informationstechnisch einfacheren Methoden der Bildverarbeitung zählt, bietet es spezifische pragmatische Vorteile. Für die hier fokussierte konzeptionelle Anwendung hat sich der Kaskadenklassifikator im praktischen Entwicklungsprozess als wesentlich anwendungsfreundlicher und ressourcenschonender herausgestellt. Aus diesem Grund wurde dieser klassische Ansatz als die primäre Methode ausgewählt und zur praktischen Anschauung im Rahmen des Versuchsaufbaus funktional umgesetzt.

5.1.2.2 Haar Cascade Classifier

Der reale Einsatzkontext des entwickelten Systems ist durch eine industrieähnliche Umgebung charakterisiert, welche inhärent eine Vielzahl optischer Störfaktoren aufweist. Hierzu zählen insbesondere teilweise stark fluktuierende Lichtverhältnisse, ein unruhiger und nicht immer idealer Hintergrund sowie das Vorhandensein anderer zu detektierender Objekte im Erfassungsbereich des Sensors. Zusätzlich erschweren transiente Schattenwürfe und unvorhersehbare Reflexionen eine konstante visuelle Erfassung. Um das Erkennungssystem unter diesen anspruchsvollen Bedingungen so robust wie möglich zu gestalten, gleichzeitig jedoch den zeitlichen und inhaltlichen Rahmen dieser Arbeit nicht zu sprengen und die vorhandene Rechenleistung des Zielsystems nicht überzustrapazieren, fiel die technologische Entscheidung auf ein Haar-Cascade-Modell. Dieses spezifische Klassifikatorenmodell wurde explizit darauf trainiert, die menschliche Hand als Zielobjekt zu identifizieren. Durch die mathematische Implementierung verschiedener, kontrastbasierter Filter (siehe Abbildung 5.2) innerhalb der Berechnungspipeline wird zudem gewährleistet, dass die Handgeometrie selbst beim Tragen von persönlichen Arbeitshandschuhen zuverlässig und resilient erkannt werden kann.

Bildverarbeitungsstruktur

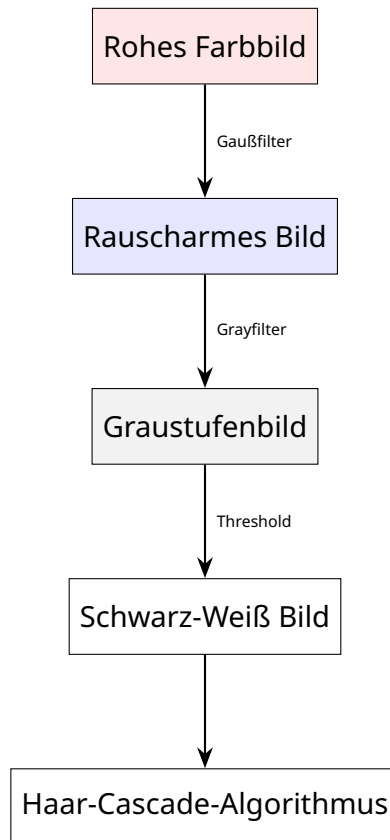


Abbildung 5.2: Struktur der Bildverarbeitung dargestellt als Blockschaltbild

5.1.2.3 Testreihe

Die Durchführung der Testreihe dient primär der systematischen Validierung und dem funktionalen Nachweis der Erkennungsrate des Modells. Hierbei wird experimentell überprüft, inwieweit das System in der Lage ist, eine menschliche Hand unter idealen Randbedingungen deterministisch zu erfassen, in welchen Grenzfällen die Detektion versagt und ob fälschlicherweise fremde Objekte als Hand klassifiziert werden (Vermeidung von *False Positives*).

Darüber hinaus bietet die Testumgebung die Möglichkeit, die zentralen mathematischen Stellgrößen des Algorithmus zu evaluieren und für eine optimale Handerkennung präzise zu parametrisieren. Zu diesen Stellgrößen gehört zum einen der Threshold-Wert der Schwellenwert-Segmentierung innerhalb der Bildvorverarbeitung,

welcher im praktischen Versuch jedoch nur eine sehr geringe Auswirkung auf das finale Erkennungsergebnis zeigte. Zum anderen lassen sich die zwei kritischsten Kernparameter des eigentlichen Haar-Cascade-Algorithmus feinjustieren: der Skalierungsfaktor (*Scale Factor*), welcher die relative Schrittweite bei der Vergrößerung des Suchfensters während der multidimensionalen Bildanalyse definiert, sowie der Parameter der minimalen Nachbarn (*Minimum Neighbors*). Letzterer legt fest, wie viele qualifizierende Detektionsrechtecke sich in unmittelbarer Nachbarschaft überlagern müssen, damit eine Bildregion final und rechtssicher als Hand deklariert wird.

Hand ohne Störeinflüsse

Um zu testen, ob der Sourcecode sowie der Algorithmus funktionieren, wurde zu Beginn eine Versuchsreihe nur mit einer Hand über dem weißen Hintergrund durchgeführt. Auch auf verschiedene Handformen wurde der Algorithmus dabei erprobt.

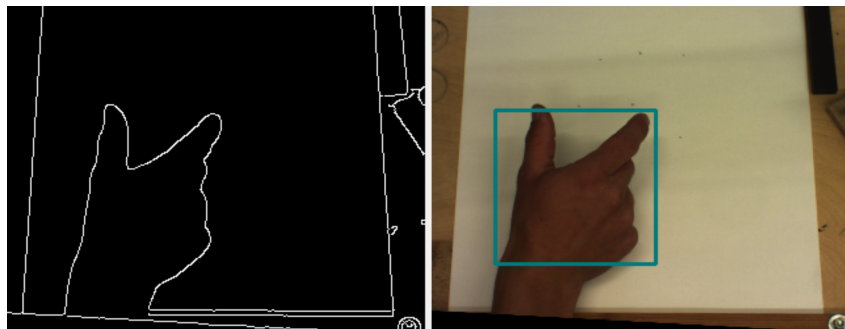


Abbildung 5.3: Testreihe (Hand allein)

Scale Factor (min. Neighbors = 6)

Die Laborversuche haben gezeigt, dass der Skalierungsfaktor (Scale Factor) – also die Schrittweite, mit der das Suchfenster des Algorithmus vergrößert wird – eine der wichtigsten Stellgrößen für die Zuverlässigkeit der Handerkennung ist. Für die Auswertung wurden drei verschiedene Parametrisierungen getestet:

- **Hoher Skalierungsfaktor (1,3 / 30% Schrittweite):** Bei dieser Einstellung wird die Hand nur dann zuverlässig erkannt, wenn sie absolut statisch und mittig im Bild positioniert ist. Sobald eine leichte Bewegung stattfindet, bricht die Detektion ab. Der Grund hierfür ist die zu grobe Schrittweite von 30% pro Skalierungsdurchlauf, wodurch der Algorithmus die sich verändernden Geometrien einer bewegten Hand überspringt.

- **Mittlerer Skalierungsfaktor (1,1 / 10% Schrittweite):** Das Verhalten ist vergleichbar mit der 30%-Einstellung. Allerdings reagiert das System bei Handbewegungen etwas toleranter und verliert das Zielobjekt nicht ganz so abrupt. Für eine dynamische Erkennung im laufenden Betrieb ist dieser Wert jedoch immer noch zu ungenau.
- **Niedriger Skalierungsfaktor (1,01 / 1% Schrittweite in Kombination mit Minimum Neighbors = 6):** Bei dieser Konfiguration wird die Hand sehr zuverlässig detektiert. Die Erkennung bleibt selbst bei schnellen Bewegungen stabil und funktioniert sogar dann, wenn die Hand nur teilweise im Erfassungsbereich der Kamera zu sehen ist. Durch die minimale Schrittweite von nur 1% scannt der Algorithmus das Bild extrem feingliedrig ab.

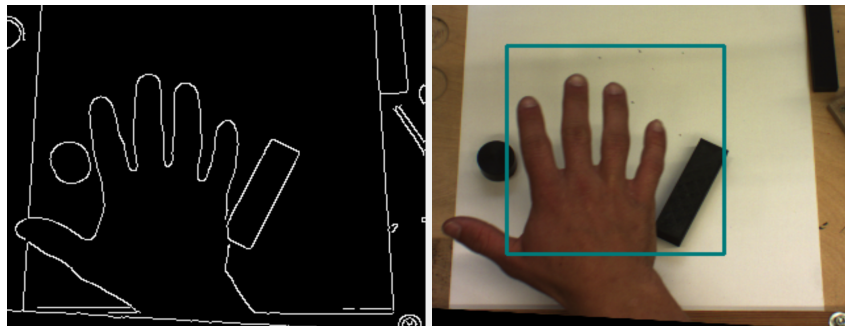


Abbildung 5.4: Testreihe (Zuverlässige Handdetektion)

Minimum Neighbors

Neben dem Skalierungsfaktor wurde im Rahmen der Testreihe auch der Einfluss des Parameters Minimum Neighbors untersucht. Dieser Wert bestimmt, wie viele überlagernde Detektionsfenster in einer Bildregion anschlagen müssen, damit der Algorithmus diese final als Hand wertet. Für die Analyse wurden drei Stufen evaluiert:

- **Hoher Wert:** Wird der Parameter zu groß gewählt, verhält sich das System extrem intolerant. Da der Algorithmus nun eine sehr hohe Anzahl an deckungsgleichen Treffern für dieselbe Bildregion erzwingt, wird die Hand nur noch in seltenen Ausnahmefällen erkannt. Für den praktischen Einsatz ist diese Einstellung daher zu restriktiv.

- **Mittlerer Wert:** Bei einer moderaten Erhöhung des Parameters bleibt die Detektion der Hand im statischen Zustand weiterhin verlässlich. Allerdings verlangsamt sich die Verarbeitungsgeschwindigkeit im Algorithmus spürbar, da pro Region mehr mathematische Übereinstimmungen geprüft werden müssen. Bei schnellen, dynamischen Handbewegungen führt dies im Versuch zu einer teilweise unzuverlässigen Erkennung, da das Objekt die Detektionszone verlässt, bevor die nötige Anzahl an Nachbarn erreicht ist.
- **Niedriger Wert (Einstellung = 1 bei einem Scale Factor von 1%):** Bei dieser Konfiguration arbeitet das System hochgradig sensitiv. Die Hand wird in fast 100% aller Fälle erfolgreich detektiert. Diese Einstellung funktioniert im Laborversuch perfekt, solange alle Bildbereiche fehlerfrei klassifiziert werden. Wie Abbildung 5.5 zu entnehmen ist, kann mit dieser Konfiguration vor allem eine zuverlässige detektion von bewegten Händen gewährleistet werden.

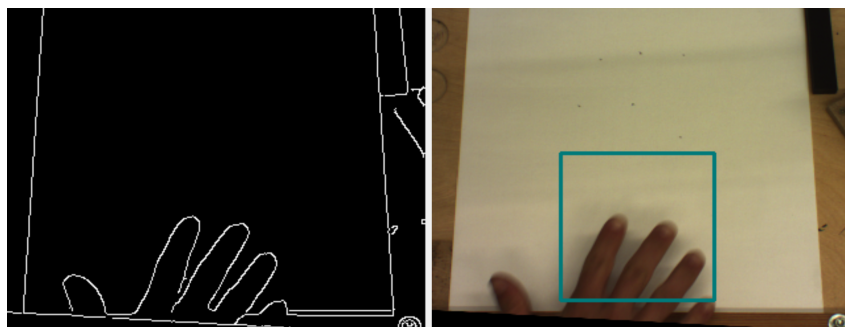


Abbildung 5.5: Testreihe (Bewegte Hand)

Der Nachteil ist jedoch eine erhöhte Anfälligkeit für Störungen: Können Hintergrundstrukturen nicht sauber abgelehnt werden, führt die geringe Schwelle dazu, dass Fremdobjekte fälschlicherweise als Hand interpretiert werden (False Positives).

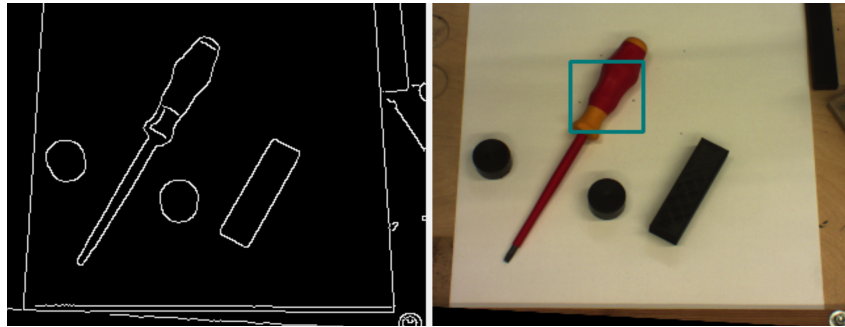


Abbildung 5.6: Testreihe (Fehlererkennung von Fremdobjekten)

Hand über Systemobjekten

Die Software sieht eine Zensur von erkannten Systemobjekten die an den Roboter weitergegeben werden vor, um Pseudoerkennung dieser Objekte auszuschließen. Kommt es jedoch zu einer teilweisen Abschirmung der Objekte durch eine Hand, so können die Systemobjekte nicht mehr vom Programm erkannt und zensiert werden. Die Konturen verschwimmen in einander und eine Handdetektion wird nahezu unmöglich.

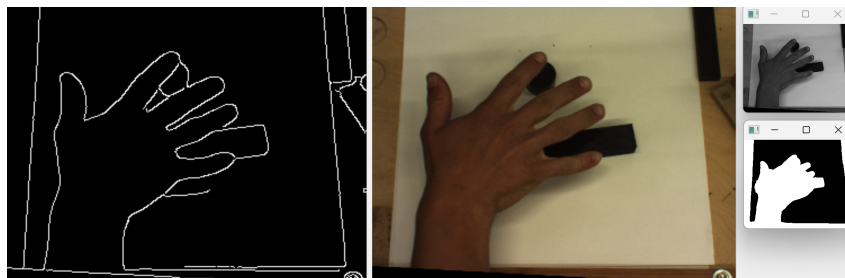


Abbildung 5.7: Testreihe (abgedeckte Systemobjekte)

Fazit

Aus der Versuchsreihe können wertvolle Erkenntnisse gezogen werden. Einerseits die Eingrenzung des Systemanwendungsbereichs, andererseits eine optimale Parametrisierung des Algorithmus für zuverlässige Ergebnisse.

5.2 Befehlspipeline

5.2.1 Vergleich von theoretischer und realer Performance

Nach der erfolgreichen Implementation der Befehlspipeline in die Steuerungsarchitektur wird das in Kapitel 3.1.2.2 eingeführte Messexperiment erneut durchgeführt. Für die bessere Vergleichbarkeit werden in MATLAB die zu erwartenden Ausführungszeiten berechnet und nachfolgend in Tabelle 5.2 dargestellt: Auch hier kommen

Sequenz	Beschreibung	Zeit [s]
1	Startposition -> Objekt 1 + GC	4,793 + 0,5
1	Objekt 1 -> Objekt 1- Z_{offset}	0,682
1	Objekt 1- Z_{offset} -> Magazin 1- Z_{offset}	1,555
1	Magazin 1- Z_{offset} -> Magazin 1 + GO	0,697 + 1
1	Magazin 1 -> Magazin 1- Z_{offset}	0,697
2	Magazin 1- Z_{offset} -> Objekt 2 + GC	2,195 + 0,5
2	Objekt 2 -> Objekt 2- Z_{offset}	0,676
2	Objekt 2- Z_{offset} -> Magazin 2- Z_{offset}	1,586
2	Magazin 2- Z_{offset} -> Magazin 2 + GO	0,638 + 1
2	Magazin 2 -> Magazin 2- Z_{offset}	0,638
2	Magazin 2- Z_{offset} -> Objekt 3 + GC	1,160 + 0,5
3	Objekt 3 -> Objekt 3- Z_{offset}	0,677
3	Objekt 3- Z_{offset} -> Magazin 3- Z_{offset}	0,886
3	Magazin 3- Z_{offset} -> Magazin 3 + GO	0,647 + 1
3	Magazin 3 -> Magazin 3- Z_{offset}	0,647
4	Magazin 3- Z_{offset} -> Objekt 4 + GC	0,915 + 0,5
4	Objekt 4 -> Objekt 4- Z_{offset}	0,634
4	Objekt 4- Z_{offset} -> Magazin 4- Z_{offset}	0,918
4	Magazin 4- Z_{offset} -> Magazin 4 + GO	0,649 + 1
4	Magazin 4 -> Magazin 4- Z_{offset}	0,649
-	Magazin 4- Z_{offset} -> Startposition	4,467
		31,88

Tabelle 5.2: Theoretische Ausführungszeiten der Befehlspipeline

erneut *Stopwatches* aus *Systems.Diagnostics* zum Einsatz um die Laufzeit zu messen. Nach fünf Durchläufen kann eine gemittelte Zeit von 32,28 Sekunden festgestellt werden - das entspricht einer Abweichung vom erwarteten Wert von ca. 1,28%. Es lässt sich erkennen, dass der Overhead beider Architekturen nahezu gleich ist.

Die Befehlspipeline ermöglicht allerdings eine Reduktion auf 31% der ursprünglichen Ausführungszeit.

5.3 Integration der Safety-Implementation in die Befehlspipeline

Um eine reibungslose und latenzfreie Kombination aus der kontinuierlichen Bildverarbeitung (Kamera-Thread) und der zeitkritischen Robotersteuerung zu gewährleisten, wurde die Not-Aus-Signalisierung nach dem Task-based Asynchronous Pattern (TAP) realisiert.

Klassische, synchrone Event-Handler bergen das Risiko, den aufrufenden Thread während der Ausführung nachgelagerter Logik zu blockieren. Da die Methode *DetectHands* zyklisch mit der Bildwiederholungsrate der Kamera angerufen wird, würde eine blockierende Event-Architektur zu massiven Verzögerungen in der Bildverarbeitung führen. Aus diesem Grund wird der Abbruchimpuls bei positiver Detektion mit dem bool'schen Rückgabewert (*pHandDetected = true*) über ein asynchrones Delegaten-Event (*Func<Task> OnHandDetectedAsync*) entkoppelt. Durch den Einsatz der Task Parallel Library (TPL) und der Methode *Task.Run* wird die Not-Aus-Aktivierung sofort in einen unblockierten Thread des .NET-ThreadPools ausgelagert.

5.3.1 Kooperativer Abbruchmechanismus mit CancellationTokenSource

Das abonnierte Event greift direkt auf die threadsichere Klasse *Robot_Befehlspipeline* zu. Der Kern des Stopps basiert auf dem Prinzip des kooperativen Abbruchs unter Verwendung einer *CancellationTokenSource* (CTS).

Der interne Worker-Task der Pipeline (*ProcessQueueAsync*) verarbeitet die Roboterkommandos sequenziell und befindet sich während der physischen Verfahrszeiten des Manipulators in einem blockierungsfreien Wartezustand, welcher durch das übergebene *CancellationToken* überwacht wird.

Sobald das asynchrone Event der Bildverarbeitung den Not-

Aus auslöst, wird die Methode *EmergencyStopAndClear()* ausgeführt. Der Ablauf sieht dabei wie folgt aus:

1. **Zustandsänderung des Tokens (*_cts.Cancel()*):** Das CTS-Objekt wechselt in den Zustand *Cancelled* und signalisiert allen konsumierenden Tasks den Abbruch.
2. **Asynchroner Interrupt:** Das aktuell aktive *Task.Delay* in der Pipeline bricht den internen Timer ab und wirft eine *TaskCancellationException*. Die Pipeline muss somit nicht das reguläre Ende des Verfahrzeit-Timeouts abwarten, sondern wechselt in den Zustand "Pipeline blockiert".
3. **Evakuierung der BlockingCollection:** Gleichzeitig wird der Befehlspuffer der Pipeline (*BlockingCollection<RobotCommand>*) geleert. Nachfolgende, bereits eingereihte Fahr- oder Greifbefehle werden somit verworfen und nicht mehr an die Hardware übermittelt.
4. **Verriegelung:** Das Flag *_isEmergencyMode* wird auf *true* gesetzt, wodurch das softwareseitige Einreihen neuer Befehle blockiert ist, bis ein System-Reset/Quittierung erfolgt.

Durch diese Kombination aus TPL-Tasks und des Werfen des *CancellationToken* wird sichergestellt, dass die Pipeline abbrechbar ist, ohne die verarbeitenden Threads (Kamera & Controller) zu blockieren oder einzuschränken.

6 Fazit & Ausblick

Im Rahmen dieser Studienarbeit konnte erfolgreich die methodische und technische Grundlage für die Transformation des vorhandenen Robotersystems - bestehend aus dem EX RV-M1 und einem Vision-System (2D & 3D-ToF) - hin zu einem limitiert kollaborativen und dynamischen System gelegt werden.

Zunächst wurden durch eine IST-Analyse die konkreten Limitationen der synchronen Softwarearchitektur, insbesondere die blockierenden Wartezeiten durch die Verwendung von Thread.Sleep()-Befehlen, quantitativ aufgezeigt. Auf Basis dieser Erkenntnisse wurde eine asynchrone Befehlsstruktur konzipiert und implementiert. Diese verwendet moderne .NET-Prinzipien (TPL und TAP) und eine prädiktive Zeitberechnung mittels inverser Kinematik, um die Laufzeit des Systems an die physisch auszuführenden Bewegung anzupassen. Diese Umstellung bietet weitere Optimierungspotenziale und Testfälle, welche in nachfolgenden Arbeiten umgesetzt werden können. Dazu zählen:

- Weitere Optimierung der Laufzeit durch Messung der real benötigten Zeit für das Senden von Befehlen sowie Öffnen/Schließen des Greifers
- Aufbrechen der MP-Befehle über Trajektoriensegmentierung um ein schnelleres Stoppen der Pipeline nach jedem gefahrenen Segment zu realisieren (vgl. Konzept in Studienarbeit T3100 Adrian Sommer & Luis Thiel)
- Grenzfallbetrachtung: Tests mit verschiedenen Objekten und Orientierungen - Zuverlässigkeit und Reproduzierbarkeit evaluieren

Des Weiteren konnte eine aktive Detektion von menschlichen Händen mithilfe der 2D-Kamera umgesetzt werden. Hierzu kommt die Machine-Learning-Methode *Haar Cascade Classifier* zum Einsatz. Dabei handelt es sich um einen vortrainierten Datensatz, welcher es ermöglicht, einen fest definierten Arbeitsbereich zu überwachen und diesen nach Merkmalen zu filtern, welche einer Hand entsprechen. Die finale Implementation wurde erfolgreich mittels einer Testreihe validiert, in welcher auch ein Ignorieren der für den Roboter relevanten Objekte gezeigt wurde.

Aufbauend auf dieser Studienarbeit ergeben sich für die Zukunft Weiterentwicklungspotenziale wie z.B. die Untersuchung von KI-basierten Safety-Ansätzen oder die Implementierung eines externen Sensor/CPU-System um einen hardwareseitigen Not-Halt in die Steuerung zu implementieren.

Literaturverzeichnis

- [1] Y. Benbelkacem and R. Mohd-Mokhtar, “Explicit kinematic model of the mitsubishi rv-m1 robot arm,” *International Journal of Advanced Robotic Systems*, vol. 8, no. 4, pp. 114–124, 2011.
- [2] Fraunhofer-Institut für Fabrikbetrieb und -automatisierung IFF, “Projektions- und kamerabasierte sicherheitstechnologien,” <https://www.iff.fraunhofer.de/de/geschaeftsbereiche/robotersysteme/projektions-und-kamerabasierte-technologien.html>, 2026, abgerufen am: 11. Juni 2026.
- [3] M. Bdiwi, M. Pfeifer, and A. Sterzing, “A new strategy for ensuring human safety during various levels of interaction with industrial robots,” *CIRP Annals*, vol. 66, no. 1, pp. 453–456, 2017.
- [4] F. Graziola, “Entwicklung eines prototypen einer passiv-infrarot schutzeinrichtung zur erkennung der hand-arm region bei sägearbeiten an einer kreissäge,” Diplomarbeit, Fachhochschule Bonn-Rhein-Sieg, Fachbereich Elektrotechnik, Maschinenbau und Technikjournalismus, Sankt Augustin, Aug. 2003.
- [5] W. Schulz, “Anwendung und vergleich von algorithmen zur sicherheitsgerichteten auswertung von 2d laserscanner-entfernungsdaten,” Sankt Augustin, 2005.
- [6] N. Mandischer, C. Weidemann, M. Hüsing, and B. Corves, “Berührungslose sicherheit für serielle manipulatoren durch optische eintrittsdetektion mit einem bewegten 3d-kamerasensor,” in *Tagungsband des Robotik-Kongresses (oder entsprechende Konferenz/Fachtagung)*. Aachen, Deutschland: Institut für Getriebe-technik, Maschinendynamik und Robotik (IGMR), RWTH Aachen University, 2022, pp. 1–8.
- [7] S. K. Singh, D. S. Chauhan, M. Vatsa, and R. Singh, “A robust skin color based face detection algorithm,” *Tamkang Journal of Science and Engineering*, vol. 6, no. 4, pp. 227–234, 2003.
- [8] Europäisches Parlament und Rat der Europäischen Union, “Richtlinie 2006/42/eg vom 17. mai 2006 über maschinen und zur Änderung der richtlinie 95/16/eg (neufassung),” Amtsblatt der Europäischen Union, L 157/24, 2006, gültig bis zum 19. Januar 2027.

- [9] —, “Verordnung (eu) 2023/1230 vom 10. mai 2023 über maschinen sowie zur aufhebung der richtlinie 2006/42/eg des europäischen parlaments und des rates und der richtlinie 73/361/ewg des rates,” Amtsblatt der Europäischen Union, L 165/1, 2023, verbindlich ab dem 20. Januar 2027.
- [10] DIN Deutsches Institut für Normung e. V., “Sicherheit von maschinen – sicherheitsbezogene teile von steuerungen – teil 1: Allgemeine gestaltungsleitsätze (iso 13849-1:2023); deutsche fassung en iso 13849-1:2023,” Beuth Verlag GmbH, Berlin, Norm DIN EN ISO 13849-1, Dec. 2023.
- [11] —, “Industrieroboter – sicherheitsanforderungen – teil 1: Roboter (iso 10218-1:2011); deutsche fassung en iso 10218-1:2011,” DIN Media GmbH, Berlin, Norm DIN EN ISO 10218-1, Jan. 2012.
- [12] —, “Industrieroboter – sicherheitsanforderungen – teil 2: Robotersysteme und integration (iso 10218-2:2011); deutsche fassung en iso 10218-2:2011,” DIN Media GmbH, Berlin, Norm DIN EN ISO 10218-2, Jun. 2012.
- [13] —, “Roboter und robotikgeräte – kollaborierende roboter (iso/ts 15066:2016),” DIN Media GmbH, Berlin, Vornorm DIN ISO/TS 15066*DIN SPEC 5306, Apr. 2017.
- [14] DIN Deutsches Institut für Normung e. V. und VDE Verband der Elektrotechnik Elektronik Informationstechnik e. V., “Sicherheit von maschinen – berührungslos wirkende schutzeinrichtungen – teil 1: Allgemeine anforderungen und prüfungen (iec 61496-1:2020); deutsche fassung en iec 61496-1:2020,” VDE Verlag GmbH / DIN Media GmbH, Berlin, Norm DIN EN IEC 61496-1 (VDE 0113-201), Jun. 2021.
- [15] DIN Deutsches Institut für Normung e. V., “Sicherheit von maschinen – anordnung von schutzeinrichtungen im hinblick auf annäherung des menschlichen körpers (iso 13855:2024); deutsche fassung en iso 13855:2024,” DIN Media GmbH, Berlin, Norm DIN EN ISO 13855, Oct. 2025.
- [16] N. Tamascelli, A. Campari, T. Parhizkar, and N. Paltrinieri, “Artificial intelligence for safety and reliability: A descriptive, bibliometric and interpretative review on machine learning,” *Journal of Loss Prevention in the Process Industries*, vol. 90, p. 105343, 2024.

- [17] APMonitor, “Machine learning for engineers: Cascade classifier,” <https://apmonitor.com/pds/index.php/Main/CascadeClassifier>, angerufen am: 21. Juni 2026.
- [18] Smarter, “Target detection: Viola-Jones detection algorithm (Haar-like features + AdaBoost + cascade classifier),” Zhihu Zhuanlan (<https://zhuanlan.zhihu.com/p/38056144>), Jun. 2018, abgerufen am: 21. Juni 2026.
- [19] W. Burger and M. J. Burge, *Digitale Bildverarbeitung: Eine algorithmische Einführung mit Java*, 4th ed. Berlin, Heidelberg: Springer Vieweg, 2022.
- [20] B. Jähne, *Digitale Bildverarbeitung: Lehrbuch und Plattform für die Praxis*, 8th ed. Berlin, Heidelberg: Springer Vieweg, 2023.
- [21] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*, 1st ed. Sebastopol, CA: O’Reilly Media, 2008.
- [22] Microsoft Developers. How to implement a producer-consumer dataflow pattern. [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/how-to-implement-a-producer-consumer-dataflow-pattern>
- [23] —. asynchrone programmiermuster. [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/asynchronous-programming-patterns/>
- [24] —. Task parallel library tpl. [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- [25] S. Cleary, *Concurrency in CSharp Cookbook*, 2nd ed. O’Reilly Media, Inc., 2019.
- [26] Microsoft Developers. Task-based asynchronous pattern (tap). [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/asynchronous-programming-patterns/task-based-asynchronous-pattern-tap>
- [27] M. Electric. Mitsubishi movemaster ex rv-m1 manual. [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: https://www.roboex.com/Mitsubishi%20Movemaster%20Robot%20Manual/RV-M1/RV-M1_InstructionManual.pdf

-
- [28] R. Meta, “Event-driven programming for responsive robotic systems,” <https://roboticsmeta.com/event-driven-programming-for-responsive-robotic-systems/>, 2024, accessed: 2026-06-19.
- [29] K. Wüst, “Grundlagen der robotik,” Skript, Technische Hochschule Mittelhessen, 2018, [Online]. Verfügbar unter: <https://homepages.thm.de/~hg6458/Robotik/Robotik.pdf> [Zugriff am 20. Juni 2026].

Abkürzungsverzeichnis

2D	Zweidimensional
3D	Dreidimensional
CLR	Common Language Runtime
Cobot	Collaborative Robot
CPU	central processing unit
CTS	CancellationTokenSource
DH	Denavit-Hartenberg
DIN	Deutsches Institut für Normung
DLL	Dynamic Link Library
DU	Drive Unit
EMV	Elektromagnetische Verträglichkeit
EN	Europäische Norm
ESPE	Berührungslos wirkende Schutzeinrichtung
FOV	Field of View
fps	frames per second (Bilder pro Sekunde)
GC	Gripper close
GO	Gripper open
GPU	graphic processing unit
GUI	Graphical User Interface (Grafische Benutzeroberfläche)
HRI	Human-Robot Interaction
HSV	Hue/Saturation/Value-Model
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LUT	Look-up-Table
ML	Maschinelles Lernen
MP	move position
MRK	Mensch-Roboter-Kollaboration
ms	Millisekunde
Mutex	Mutual Exclusion
NPU	Network processing unit
OEE	Overall Equipment Effectiveness
OG	Origin
PCL	Point Cloud Library
PFL	Power and Force Limiting
PIR	Passiv-Infrarot-Sensoren
PL	Performance Level
RGB	Rot-Grün-Blau

ROI	Region of Interest
RS-232	Recommended Standard 232
SAT	Summed-Area Table
SLS	Safely-Limited Speed
SP	Speed
SSM	Speed and Separation Monitoring
TAP	Task-based Asynchronous Pattern
TCP	Tool Center Point
TCP/IP	Transmission Control Protocol/Internet Protocol
ToF	Time of Flight
TPL	Task Parallel Library
WH	Where (Befehl zur Positionsabfrage)
YCbCr	Farbmodell für Bild- und Videoaufzeichnungen
YOLO	You only look once

Abbildungsverzeichnis

2.1	Testbild Passiv-Infrarot Schutzeinrichtung an einer Kreissäge aus [4]	5
2.2	Prinzip-Skizze Haar-like-features aus [18]	10
2.3	Architektur eines Producer-Consumer-Patterns (eigene Darstellung)	15
2.4	Problematik von blockierenden Abläufen (eigene Darstellung)	15
2.5	Kinematische Parameter des RV-M1	18
2.6	Skizze Denavit-Hartenberg-Frames des RV-M1 [1]	18
3.1	Sequenzielle Abfolge an Befehlen (Eigene Darstellung) . .	22
3.2	Messexperiment zur Evaluation der Sortierperformance (eigene Darstellung)	24
3.3	Befehle innerhalb einer Objektsortierungssequenz (eigene Darstellung)	25
4.1	Bewegungsablauf asynchrone/synchrone PTP nach [29] .	30
4.2	Vergleich zwischen monolithischer Befehlsverarbeitung und entkoppelter Pipeline-Architektur	31
5.1	Projektions- und kamerabasierte Technologie zur Arbeitsraumüberwachung	35
5.2	Struktur der Bildverarbeitung dargestellt als Blockschaltbild	40
5.3	Testreihe (Hand allein)	41
5.4	Testreihe (Zuverlässige Handdetektion)	42
5.5	Testreihe (Bewegte Hand)	43
5.6	Testreihe (Fehlerkennung von Fremdobjekten)	44
5.7	Testreihe (abgedeckte Systemobjekte)	44

Erklärungen zur Arbeit und Hilfsmitteln

Verwendete Hilfsmittel

Für die Erstellung dieser Studienarbeit wurden folgende Tools eingesetzt:

- **Hardware:** SICK Visionary-T Mini-CX, Mitsubishi Movemaster EX-RV M1, uEye Industriekamera
- **Software:** OpenCV Bibliothek, Visual Studio (C++ und C#), RoboETH, drawIO (Erstellung von Grafiken), texStudio, uEye-Cockpit und IDS Kameramanager, GitHub (Kollaboratives Arbeiten und Versionsverwaltung der LaTeX-Vorlage), MATLAB 2025, GitHub CoPilot.

KI- Tools und relevanter Einsatzbereich

Dies erfolgt im Einklang mit der entsprechenden Verordnung der DHBW (Weblink).

Tool	Art der Nutzung	Abschnitt
Google Gemini AI Student Pro License	Formatierung von LaTeX-Code, Erstellung von Blankovorlagen für Tabellen und Listen	gesamte Arbeit
DeepL AI Übersetzer	Formulierung des englischsprachigen Abstracts	Abstract
Github CoPilot mit Gemini API	Programmiersyntax und Formatierung der Projektmappe, IntelliSense-Erweiterung, Refactoring der Klassen "Roboter.cs, Befehlspipeline.cs, Controller.cs und Bildverarbeitung.cs	Visual Studio-Projektmappe

Aufteilung der Kapitel

Die Erstellung der vorliegenden Studienarbeit erfolgte in gemeinsamer Kooperation. In der folgenden Tabelle ist dokumentiert, welcher Autor die primäre Urheberschaft für die jeweiligen Kapitel bzw. Abschnitte trägt:

Kapitel / Abschnitt	Kapitelnummer	Autor
Einleitung	1	Gemeinsame Bearbeitung
Safety-Implementation	2.1	Luis Magnus Thiel
Bildverarbeitung	2.2	Luis Magnus Thiel
Grundlagen der Informatik	2.3	Adrian Sommer
Inverse Kinematik	2.4	Adrian Sommer
Verwaltung von Punktwolken	3.1.1	Luis Magnus Thiel
Sequenzielle Befehlsstruktur	3.1.2	Adrian Sommer
Konzeptionierung der dynamischen Robotersteuerung	4.1	Adrian Sommer
Safetyimplementation	5.1	Luis Magnus Thiel
Befehlspipeline	5.2	Adrian Sommer
Integration der Safety-Implementation in die Befehlspipeline	5.3	Adrian Sommer
Fazit und Ausblick	6	Gemeinsame Bearbeitung

Hiermit bestätigen wir, Luis Magnus Thiel und Adrian Sommer,
dass die oben gemachten Angaben der Wahrheit entsprechen.

Horb, den 22. Juni 2026



Ort, Datum

Unterschrift (Luis Magnus Thiel)

Horb, den 22. Juni 2026



Ort, Datum

Unterschrift (Adrian Sommer)

Anhang

Inhalt des digitalen Anhangs

- Änderung zu PCL-Verwaltung (Luis Thiel)
- gesamter LaTeX-Ordner
- MATLAB-Livescript für die Berechnung der Objekt-Verfahrzeiten
- relevante .cs-Dateien (Roboter, Befehlspipeline, Controller, Bildverarbeitung)
- Link zur vollständigen Projektmappe